Retainable Interactive Computation: Operational Semantics, Algebraic Effects, and Crash-Consistent Recovery in the Agentic Lambda Abstraction Network (ALAN)

ALAN Research Monograph

October 2026

Abstract

What must survive a process crash for an agent waiting on an external reply to continue? The Agentic Lambda Abstraction Network (ALAN) retains a versioned program, its next instruction and typed state, and the identities of outstanding requests. A model response enters the computation as an observation. Local steps are deterministic, and a separate authority decision governs whether the runtime may dispatch a proposed mutation. The single-host runtime stores this state in SQLite and can replay committed steps without contacting an endpoint.

Ordinary programs can supervise children and collect their outcomes. In the family of finite trees studied here, each parent owns its children. One leaf program computes locally and another calls a helper. On the supported inputs, substituting one for the other preserves which answers and failure diagnoses reach the caller and when it must keep waiting. The proof assumes an open tree, stable authority, sufficient shared resources and eventual service of eligible work.

If a crash leaves a non-idempotent request without a recorded receipt, the local journal can be identical whether the endpoint acted or the request was lost. In a fixed composition, authorized readers can obtain evidence for two original mutations from a trusted application ledger. Independent finite models and process-crash experiments test selected instances. General guarantees that substitution and recovery preserve future behavior and exact resource use remain open.

Contents

1 Introduction: The Ontological Crisis of Agentic Computation **3**
- 1.1 Orchestration Frameworks versus Models of Computation . . . 3
- 1.2 Durable Workflows versus Durable Computation . . . 3
- 1.3 Distinguishing Engineering Synthesis from Mathematical Contributions . . . 4

2 Pillar I: Computational Universality, Bounded Realizability, and Felleisen Expressiveness **5**
- 2.1 Beyond the Turing Tarpit . . . 5
- 2.2 Bounded Supervision and a Ledger-Only Boundary . . . 6
- 2.3 Bounded Data-Dependent Iteration and Snapshot Cursors (Phase 1G) . . . 9

3 Pillar II: Structural and Mathematical Operational Semantics **11**
- 3.1 Small-Step Operational Semantics of Pure Local Reduction (δ_v) . . . 11
- 3.2 Global Transition Rules at the Runtime Boundary . . . 11

4 Pillar III: Dynamic Concurrency, Orthogonal Authority, and Compositional Congruence **11**
4.1 The Four-Tier Identity Hierarchy and Orthogonal Authority Graphs 11
4.2 Two-Tier Injective Identity Algebra 12
4.3 Bounded Template Invocations and Conditional Congruence 13

5 Pillar IV: Algebraic Effects, Split-Phase Operational Handlers, and the Epistemic Effect Lattice **24**
5.1 Split-Phase ITree Lowering versus the Asynchronous Handler Algebra 24
5.2 Separating Probabilistic Intelligence from Deterministic Execution 28

6 Pillar V: Crash-Consistent Operational Semantics and the Two-Regime Fundamental Theorem **28**
6.1 Two-Tier Configurations, the Recovery Operator, and the Window-W_3 Impossibility Theorem 28
6.2 The Two-Regime Fundamental Theorem of Durable Computation 28
6.3 Two-Agent Interrupted Delegation: the Phase 3A Custody Boundary 31
6.4 Reciprocal Obligations: the Phase 3B.1 Service Boundary 32

7 Conclusion **34**

1 Introduction: The Ontological Crisis of Agentic Computation

1.1 Orchestration Frameworks versus Models of Computation

The rapid proliferation of large language models (LLMs) has catalyzed an industry of software frameworks purporting to build "autonomous agents." Viewed through the lens of programming-language theory, however, nearly all contemporary agent frameworks are variations on a single, fragile idiom: *imperative host-language orchestration*. In such systems, an "agent" is a Python, TypeScript, or Go closure that loops over a mutable prompt buffer, invokes an external HTTP endpoint to sample tokens from a probabilistic model, parses tool-call strings from the completion, and executes arbitrary host callbacks in-place.

This architecture suffers from a foundational ontological confusion by conflating three layers that must be kept mathematically distinct:

1. **Probabilistic Epistemic Proposals:** A language model completion is not a computation step; it is an untrusted, nondeterministic external observation sampled from a stochastic oracle.

2. **Deterministic Residual Control:** The logic that validates an observation against a schema, updates local state, enforces budget ceilings, and selects the next branch of work is a pure, finite state transition.

3. **Authoritative World Transitions:** Admitting a message to a peer's inbox, mutating an external database, or incrementing the execution generation of a failed child computation are irreversible distributed commitments subject to partial failure, message reordering, and process death.

When a conventional agent framework embeds all three layers inside the call stack of a live OS process, the *continuation* of the agent—what remains to be done, which obligations are outstanding, and what authority the agent holds—is implicitly encoded in ephemeral instruction pointers, heap pointers, and open TCP sockets. If the executor process receives `SIGKILL` after dispatching an external tool request but before recording its receipt, the residual computation vanishes.

1.2 Durable Workflows versus Durable Computation

Even industrial "durable execution" engines (such as Temporal) address process crashes via event-sourced replay of host-language coroutines rather than reifying the computation itself. A decisive gulf separates *durable workflow bookkeeping* from *durable computation*:

- **First-Class Projections vs. Opaque Coroutines:** Workflow engines treat the program as an opaque host closure whose state is invisible except by re-executing host code from step 0. ALAN represents every computation as an explicit, version-bound instruction array P_v and serializable projection $\sigma = \langle pc, \rho, status, \kappa \rangle$.

- **Ordinary supervision.** The bounded supervisor consumes authenticated terminal notices and sends lifecycle intents through the same interpreter as the worker (`executor.Reduce`). The static deployment provisions separate grants for messages, terminal notices and lifecycle operations.

- **Split-Phase Effect Authority.** Intent commitment, first source permission and reservation precede worker launch and world application. Result recording, settlement and selection also have separate boundaries. Source closure before first permission suppresses the original mutation under trusted dispatch. An allowed worker can first launch after

cancellation. A current-authority gate orders admission against revocation. A separate local acknowledgment drains admitted work. An admitted worker can still enter its physical call after revocation.

Foundational Work	Fundamental Contribution	Connection to ALAN
Church (1936) [5]; Turing (1936) [20]	λ-calculus and universal mechanical computation	Small, universal compositional core without privileged agent classes
Felleisen (1991) [6]	Rigorous distinction between computability and macro-expressiveness	A bounded retry construction and a separation result for silent child outcomes at a fixed boundary
Plotkin (1981) [15]	Structural Operational Semantics (SOS)	ALAN's finite interpreter (`Reduce`) and global ledger transition rules
Turi & Plotkin (1997) [19]	Mathematical operational semantics & bialgebraic congruence	Compositional equivalence for nested agent networks
Milner, Parrow & Walker (1992) [10]; Hewitt (1973) [7]	Mobile processes (π-calculus) and actor configurations	Dynamic template invocation (`compose`), capability scoping, and identity
Moggi (1991) [12]; Plotkin & Pretnar (2009/2013) [16]	Computational monads and algebraic effect handlers	Separating pure reduction from durable intents and runtime adapter execution
Xia et al. (2020) [21]	Coinductive Interaction Trees (ITree)	Mathematical analogue to ALAN's retained interactive computations
Lamport (1978) [9]; Chandy & Lamport (1985) [4]	Causal happens-before ordering and consistent global cuts	Per-run observation serialization, generation routing, and replay cuts
Schneider (1990) [18]	Fault-tolerant deterministic state machines	Journaled replay and crash recovery extended to external uncertainty

Table 1: The intellectual lineage of ALAN across programming languages and distributed systems.

1.3 Distinguishing Engineering Synthesis from Mathematical Contributions

To situate this work precisely for a programming-languages audience, we distinguish what ALAN synthesizes from existing literature from where existing formalisms break down:

1. **Engineering Synthesis of Known Theory:** Representing local step execution (`Reduce`) as a deterministic transition function δ_v instantiates Plotkin's SOS [15]; lowering synchronous requests $\mathsf{Vis}(e, k)$ into split-phase Emit and $\mathsf{Receive}$ instructions is a standard free-monad homomorphism $\mathcal{S} : \mathsf{ITree}_{E,R} \to \mathsf{ITree}_{E_{\text{split}},R}$ [21]; and re-evaluating history rows under monotonically increasing epoch numbers $e \mapsto e + 1$ synthesizes Schneider's state-machine replay [18] and Burrows's fencing tokens [3].

2. **Specific Mathematical Contributions of This Work:**

   - *The Split-Phase Journaled Handler Algebra ($\mathcal{H}_{journal}$) and Consistent Epistemic Poset ($\mathbb{O}_{eff}^{cons}$):* Standard ITree interpretation (`interp`) applies a monadic handler $h : E_{\text{split}} \rightsquigarrow \mathsf{StateT}(\mathcal{D}, M)$ atomically inside the evaluation thread, which cannot model an effect in flight across a process crash or concurrent `CancelRun`. We formulate the asynchronous handler protocol over the consistent evidence poset $\mathbb{O}_{\text{eff}}^{\text{cons}}$

completed by $\top_{\text{conflict}}$ into $\widehat{\mathbb{O}}_{\text{eff}}$ (where $\mathsf{Suppressed}(r)$ and $\mathsf{Known}(v)$ are strictly incomparable consistent elements whose join is $\top_{\text{conflict}}$) and establish source suppression before first permission (Theorem 5.1). Phase 2A falsifies the former physical-launch clause.

- *What a bounded supervisor must observe.* The fixed inner program uses an authenticated child outcome, a budget and an original retry decision (Theorem 2.3). A two-sided induction at a ledger-only boundary separates even zero-retry outcome propagation from ordinary parent computation. A current-Store experiment shows why program-issued lifecycle requests fall outside that induction.
- *Inductive Congruence under Dual Refusals and Shared External Effects:* We show why standard weak bisimulation fails under hiding in the presence of asynchronous mailbox admission vs. synchronous continuation consumption, shared external endpoints $\mathcal{W}_\ell$, and shared root conservation envelopes $\mathcal{B}(r)$, and formulate an open conditional-congruence claim over $\mathcal{C}_{\text{adm}}$ (Theorem 4.5), with interference, invocation and scheduler obligations still to be discharged.
- *Impossibility of Unconditional Crash Bisimulation and the Two-Regime Recovery Theorem:* We prove that no deterministic recovery operator without out-of-band world introspection can achieve symmetric forward bisimulation across an unreconciled non-idempotent Window-W_3 crash while guaranteeing at-most-once world mutation (Proposition 6.1), and distinguish conservative recovery safety from the stronger symmetric and directed simulation obligations (Theorem 6.2). The general peer/interference and reconciliation arguments remain open.

2 Pillar I: Computational Universality, Bounded Realizability, and Felleisen Expressiveness

2.1 Beyond the Turing Tarpit

Church [5], Turing [20], and Minsky's Theorem Ia [11] establish that two unbounded integer registers with increment-and-jump and decrement-and-branch-on-zero suffice to compute any partial recursive function. In an open, durable agent system, however, appealing to Turing completeness obscures two critical realities:

1. **Bounded Realizability:** Practical durable deployments enforce explicit resource ceilings $\mathcal{L} = (\mathit{MaxProgramBytes}, \mathit{MaxDataBytes}, \mathit{MaxObservationBytes}, \mathit{MaxOutputBytes})$ (each 1 MiB by default in `internal/executor/program.go`) and checked 64-bit integer arithmetic (`int64` overflow trapping to `ErrLimit`). Unbounded host callbacks violate finite-step scheduling.

2. **Computability vs. Macro-Expressiveness:** Following Felleisen [6], whole-program simulation via Gödel numbering is vacuous for language design because it destroys concurrency, local state isolation, and observational equivalence in contexts.

Definition 2.1 (Contextual equivalence and macro-eliminability for an interactive observation interface)**.** Let $\mathcal{L} = (\mathsf{Syn}_{\mathcal{L}}, \mathsf{Conf}_{\mathcal{L}}, \longrightarrow_{\mathcal{L}}, \mathsf{Obs}_{\mathcal{L}})$ be an interactive calculus. Two terms $P, Q \in \mathsf{Syn}_{\mathcal{L}}$ are *contextually equivalent*, written $P \simeq_{\mathcal{L}} Q$, if $\mathsf{Obs}_{\mathcal{L}}(C[P]) = \mathsf{Obs}_{\mathcal{L}}(C[Q])$ for every closing context $C[\cdot]$. Following Felleisen [6], consider an extension $\mathcal{L}^+$ with one new k-ary constructor F. Fix a public observation interface for both languages. We call F *macro-expressible* in $\mathcal{L}$ ($\mathcal{L}^+ \preceq_{\text{macro}} \mathcal{L}$) if a syntactic translation $\mathcal{T}$ fixes core primitives, acts homomorphically on core constructors and maps $F(t_1, \ldots, t_k)$ to a fixed core context $\Phi_F[\mathcal{T}(t_1), \ldots, \mathcal{T}(t_k)]$. For each closed P, require $\mathsf{Obs}_{\mathcal{L}^+}(P) = \mathsf{Obs}_{\mathcal{L}}(\mathcal{T}(P))$. For nondeterministic executions, compare the sets of possible observations under the fixed interface. Our interactive criterion also requires

preservation and reflection of contextual equivalence. For all P, Q, $P \simeq_{\mathcal{L}^+} Q$ if and only if $\mathcal{T}(P) \simeq_{\mathcal{L}} \mathcal{T}(Q)$, with closing contexts quantified in their respective languages. If no translation $\mathcal{T}$ meets these conditions, write $\mathcal{L} \prec_{\text{macro}} \mathcal{L}^+$.

Theorem 2.2 (Inexpressibility of `Conditional` in Trace-Fixed Total `OneShot` Calculi)**.** *Let $\mathcal{L}_{fixed}$ be generated by atomic `OneShot` leaves L_i and sequential composition $P; Q$, where each leaf L_i is total, terminates normally without input-dependent failure or early exit, and emits a fixed nonempty sequence of observable effect labels $\vec{\alpha}(L_i) \in \mathsf{Act}^+$ independent of input data ρ. Let $\mathcal{L}^+_{fixed}$ adjoin $\mathsf{Conditional}(p, A, B)$ for a non-constant predicate $p(\rho) \in \{\mathsf{true}, \mathsf{false}\}$. Then $\mathcal{L}_{fixed} \prec_{macro} \mathcal{L}^+_{fixed}$.*

Proof. By structural induction on $M \in \mathsf{Syn}_{\mathcal{L}_{\text{fixed}}}$, totality and normal termination of every leaf L_i ensure that in $P; Q$, P always emits its complete fixed trace $\vec{u}$ and passes control to Q, which emits its complete fixed trace $\vec{v}$, making the concatenated trace $\pi(M, \rho) = \vec{u} \cdot \vec{v}$ independent of ρ. By contrast, $\mathsf{Conditional}(p, L_{\text{left}}, L_{\text{right}})$ with $\vec{\alpha}(L_{\text{left}}) \neq \vec{\alpha}(L_{\text{right}})$ emits input-dependent traces. (When leaves instead evaluate a pure reducer δ_v that selects residual control, `Conditional` is locally macro-eliminable via `branch` and `jump`.) □

2.2 Bounded Supervision and a Ledger-Only Boundary

Theorem 2.3 (A bounded supervisory policy)**.** *In the static `NestedProvision` deployment, the inner program implements a bounded retry policy with the seven core instructions. After a start message it sends work to the worker. An authenticated success for the expected generation ends the inner run successfully. On authenticated failure it issues one `Retry` if budget remains and otherwise ends in failure. Acceptance directs work to the reported successor. Refusal, child cancellation and child abandonment each lead to a failed inner End. These conclusions describe the serviced branches of the program.*

Proof. The outer, inner and worker subjects are distinct, initially at generation and ownership revision 1. The deployment provisions the two message routes, both terminal-observation routes and the inner subject's `Retry` grant. Assume trusted admission, atomic Store commits, an inner run kept open until its policy ends and sufficient byte, generation and sequence capacity for the steps considered. The private registers hold `spent` $= 0$, `limit` $= B$ for a nonnegative representable signed 64-bit budget, `expected` $= 1$, `revision` $= 1$, an empty `pending` and one configured successor-input string. The first work payload is fixed to `fail`. The worker compares its input with `success`. In the listing `obl` denotes the provisioned `worker-obligation`. See `internal/executor/examples.go`.

```
pc 0: receive(kind="message", prov="ordinary-message", src=Outer, gen=1, obl="inner-start") ->
    [started]
pc 1: emit(kind="message", target=Worker, gen=Ref("expected"), obl="worker-work", payload={mode
    : "fail"})
pc 2: receive(kind="terminal", prov="runtime-terminal", src=Worker, gen=Ref("expected"), obl=
    obl) -> [outcome]
pc 3: branch(Ref("outcome") == "succeeded", target=13, otherwise=4)
pc 4: branch(Ref("outcome") == "failed", target=5, otherwise=14)
pc 5: branch(Ref("spent") == Ref("limit"), target=14, otherwise=6)
pc 6: emit( dst="pending", kind="lifecycle", target=Worker, gen=Ref("expected"), obl=obl,
           payload={"operation": "Retry", "revision": Ref("revision")},
           updates={"spent": Offset("spent", +1)} )
pc 7: receive(kind="lifecycle", prov="runtime-lifecycle", src=Worker, gen=Ref("expected"),
             obl=obl, op=Ref("pending")) -> [accepted, reason, successor, disposition]
pc 8: branch(Ref("accepted") == true, target=9, otherwise=14)
pc 9: set(dst="expected", A=Ref("successor"))
pc 10: set(dst="pending", A="")
pc 11: emit(kind="message", target=Worker, gen=Ref("expected"), obl="worker-work", payload={
    mode: Ref("successor-input")})
```

```
pc 12: jump(target=2)
pc 13: end(outcome="succeeded")
pc 14: end(outcome="failed")
```

On each entry to instruction 2, `expected` names the generation to which work was last emitted and $0 \leq$ `spent` $\leq B$. Instruction 1 establishes this invariant. Instructions 3–5 permit a retry only on failure with `spent` $< B$. At instruction 6 the Store commits the intent, its original operation ID in `pending`, and the increment of `spent` together. A physical reactivation retains all three. Instruction 7 selects a runtime lifecycle result for this requester run, worker, expected generation, obligation and saved ID. On acceptance, instructions 9–12 reestablish the invariant with the reported successor. On refusal, instruction 8 branches to failure. The terminal guard at instruction 2 rejects stale generations. A delayed or withheld observation can leave the program waiting. Completion also requires service of the eligible local and protocol steps. □

The executor tests exercise successor success, budget exhaustion and reactivation with a pending request. Phase 1K separately exercises an authentic terminal receive in a raw artifact through the public runtime. The public author constructors expose a smaller vocabulary. For full macro-eliminability of a parameterized `Supervise`, one would still need source rules for its work arguments, hygienic private registers and obligations, and correspondence for every admitted context and schedule, including waits and refusals, with preservation and reflection of contextual equivalence. See `internal/executor/supervision_test.go` and the Phase 1K qualification (`docs/foundational/phase1k/02-qualification.md`).

Lemma (finite parent traces at a ledger-only boundary). Fix a deployment with a named parent p and one unmodified child. Only the parent program may be translated. At this boundary the parent has `set`, `add`, `branch`, `jump`, `receive`, `emit` and `end`. Its effects can send ordinary messages or call endpoints whose responses depend only on their own state and requests. Terminal delivery writes only to a private ledger. Only the environment can change another run's lifecycle, and it issues no lifecycle command here. This exhausts the parent's channels. The deployment has no invocations or host queries. Public inputs, grants, limits and initial endpoint state agree. Untimed scheduling and service choices are independent of the child's private input.

The pre-1C model supplies the ledger-only terminal and environmental retry rules for this comparison. Phase 1C added both terminal inbox delivery and program-issued lifecycle requests. Its terminal regression checks inbox admission after a child fault. See `research/semantics/supervision_contracts_test.go`.

In $C_b[P]$, the child has a private bit b before the parent P starts. A private startup receive can supply it in the historical model. It waits for a fixed work message, ignores its payload, branches on b and immediately ends successfully for 0 or with failure for 1. It emits nothing and owns no pending intent to suppress at closure. Both raw End instructions validate. Each intermediate and terminal child projection fits the byte limit for its full serialization, and each admitted envelope fits its own bound. The branches take the same number of steps, leave the same admission windows open, and incur the same shared debits. Let Tr_p project finite executions to the named parent's emitted intents, blocked receives and terminal status. For every well-formed parent program P at this boundary,

$$\mathrm{Tr}_p(C_0[P]) = \mathrm{Tr}_p(C_1[P]).$$

Proof. Pair the initial configurations on the parent artifact, projection, run identity, epoch, revision, sequence, mailbox including payloads and consumption flags, and remaining limits. Pair public endpoint and transport state and the children's phase and openness. Induct on a finite execution prefix. In a parent step, `set` and `add` evaluate identical operands and checks, `branch` and `jump` choose the same PC, and `receive` can select the same eligible envelope and

bind the same fields, or wait on both sides. At `emit` the descriptor and updates agree, and the same run and sequence yield the same operation ID. At `end` the parent commits the same status. Local refusals and limit failures also match.

For a world step, couple the same public input, endpoint response or service choice. If the parent withholds work, both children wait. Otherwise match their work receive, branch and End at corresponding cuts. Even a parent that probes the child with further sends sees the same generation, openness and admission capacity at each cut, hence the same admission receipts. Terminal delivery changes only the hidden ledger. The same construction matches every step in the reverse direction. Projection therefore gives equal sets of finite parent traces, although the two global journals record different child outcomes. See the pre-1C model at `82dc5649`. □

With zero retries, the specified supervisor has a serviced finite execution ending in parent success for $b = 0$, and parent success is impossible for $b = 1$. A uniform parent translation must preserve both facts. Equal trace sets would either omit the serviced success or admit it in the failed-child case. Waiting forever omits the required finite trace.

Scheduler check. In the ordinary Store, `selectReady` rotates over ordered run IDs and skips both closed outcomes. It scans observation IDs for eligible receives. End writes a separate terminal record. Terminal delivery inserts the corresponding observation. In a two-run fixture, ten matched cuts gave the same enabled parent commits, including a blocked receive and two inputs eligible at once. Servicing a granted terminal delivery then let the parent distinguish the outcomes. A separate byte-limit control fits the failed terminal projection but excludes the larger successful one. In that arm `Run` returns the child's size error while the parent still has a legal commit. The difference concerns concrete scheduler progress outside the lemma's size premise. See `internal/executor/runner_silent_child_test.go`.

Authority to create a successor. For a positive retry budget there is a separate authority requirement. At the restricted boundary only the environment's retry transition advances the child's generation. Induction on the permitted local and world steps therefore leaves it at 1. Even a parent that knew the child had failed could not create the prescribed successor there. At a boundary that permits requests, if acceptance and refusal are possible from the same parent view, the policy must also distinguish their committed decisions. Emitting successor work on refusal adds an observable intent, while escalating on acceptance omits one. An authenticated service with a correlated reply could supply this distinction.

A probe outside the lemma's boundary. After an unmodified silent child ends, schedule the same authorized `Retry` for target generation and revision 1. Success yields `accepted=false`, `not_retryable` and no successor. Failure yields `accepted=true`, `retried` and generation 2. The same ordinary parent consumes only the settlement for its saved operation ID. It branches on `accepted` alone and ends successfully after refusal or with failure after acceptance. The Store regression exercises both outcomes with a configured terminal delivery withheld and again with no terminal-observation grant. In the latter runs the Store rejects an attempted terminal delivery. The lifecycle settlement still reaches the requester. See `internal/executor/supervision_test.go`.

The ledger-only lemma identifies the information the zero-retry policy needs on the silent child. Some channel must distinguish the committed outcomes in the parent's public view. ALAN's canonical channel is the authenticated terminal envelope for the expected child generation. A retry probe distinguishes the two post-End cases above, but it creates a generation on failure and an earlier probe can mistake an open worker for a successful one. Outside the lemma, a cooperative child could send its intended outcome before `End`. Even if the message arrives, cancellation before `End` can prevent that outcome from committing. Any substitute

must supply the distinctions the policy needs on its allowed schedules and preserve its effects and retry budget.

2.3 Bounded Data-Dependent Iteration and Snapshot Cursors (Phase 1G)

Theorem 2.4 (Laminar Cursor Conservation and Strong Termination)**.** *Let P be any program passing `validateProgram(P)` under `CollectionSemanticVersion` (`internal/executor/collections.go`), with code length $L = |P.\texttt{Code}|$, maximum loop-snapshot capacity $M = \max_c c.\texttt{MaxItems}$, where c ranges over the `Collection` schemas of `foreach` instructions (set $M = 0$ if there are no loops), and actual finite loop nesting depth d. The public authoring compiler additionally enforces $d \leq \texttt{author.MaxNesting}$; the raw program validator does not impose that cap. For any valid open projection σ_0:*

1. ***Snapshot Invariance:*** *Mutations to $\rho[\texttt{items}]$ inside a loop body leave the active cursor's cloned $\kappa[pc_{start}]$.Snapshot invariant.*
2. ***Cursor Well-Bracketedness:*** *At every reachable open projection σ, $\kappa[pc_{start}]$ exists iff $pc_{start} < \sigma.pc < P.\texttt{Code}[pc_{start}].\texttt{Otherwise}$. Executing `end` clears all cursors, even when its program counter remains inside a loop interval.*
3. ***Strong Termination Bound:*** *Every sequence of successful local reductions has length at most $L \cdot (M+2)^d$. A maximal sequence ends with terminal status, a blocked `receive`, or a rejected reduction. Excluding rejection additionally requires that all executed operands, schemas, resource checks, and successor program counters are valid; `validateProgram(P)` alone does not establish this assumption. The bound counts successful instruction reductions, not unsuccessful receive polling or elapsed external waiting.*

Proof. (1) `foreach` stores a deep clone of the evaluated source list in its cursor. Subsequent `next` reductions read only this snapshot, clone it forward, and increment its index; body mutations to the source variable do not change it.

(2) `validateCollections` enforces laminar `foreach`–`loop` intervals, a matching `next` immediately after each header, and a matching `loop` immediately before its exit. It forbids backward `jump`/`branch` targets and entry from outside into a loop body. Thus a newly entered body has a cursor created by its `foreach`; an initially active loop has one by `validateCursors`. On an outward forward exit, `cleanCursors` deletes every exited cursor. It also clears all cursors on `end`, so the interval equivalence is asserted only for open projections.

(3) On a stretch of forward instructions, the natural-valued distance $L - \sigma.pc$ strictly decreases. The former coordinate $\sigma.pc$ had the wrong sign. Replacing it in the active-cursor tuple does not give a ranking that decreases at every instruction: `loop` jumps backward without changing any cursor index, and the following `next` increments it in a separate reduction. In the five-instruction program `foreach; next; set; loop; end`, with a one-element snapshot already bound, `set` at $2 \to 3$ increases the old tuple $(0, 2) \to (0, 3)$, while `loop` at $3 \to 1$ increases the sign-corrected tuple $(0, L-3) = (0, 2) \to (0, L-1) = (0, 4)$. Loop entry and exit also change the active tuple's dimension. We therefore use a structural instruction-count argument rather than assert a global per-step lexicographic rank.

For one activation with immutable snapshot length $n \leq M$, `next` binds at most n elements and performs at most one final exhaustion check; `loop` executes at most once per bound element. Induct on laminar loop depth: at depth zero all nonterminal steps advance the program counter, so each instruction executes at most once. At greater depth, the induction hypothesis makes each strictly nested loop activation finite; after collapsing those activations, each body pass contains only forward control flow up to the enclosing `loop` or an outward exit. Hence there are at most M body passes and $M + 1$ executions of `next` per activation. Outward branches and early `end` only shorten the traversal.

For an initially active loop, starting at `next` leaves at most $n - \mathit{Index}$ bindings. Starting inside its body requires $\mathit{Index} \geq 1$ by `validateCursors`, so the current partial pass plus the remaining passes number at most $1 + n - \mathit{Index} \leq n$. Each nested header is encountered at most once per enclosing body pass. Applying the bounds from outermost to innermost gives at most $(M+1)^d$ executions of any fixed instruction, including exhaustion checks. Summing over the L instructions yields at most $L(M+1)^d \leq L(M+2)^d$ successful reductions. Failed checks and waiting without an eligible observation add none. This proves the bound without claiming successful completion or a bound on external waiting. □

Bounded adaptive application (Phase 1H). `examples/adaptive` uses the existing public authoring operations to accept seed work after initialization and derive each ordered successor frontier from the preceding round's returned outcomes. A single retained three-token `ForEach` controls at most three rounds, with at most eight application-accepted leaf items across the obligation. Each round emits its item invocations, consumes all required outcomes through their original operation handles, checks the complete candidate frontier for identity collisions and remaining capacity, and retains a one-row decision before emitting successor work. Application work IDs are distinct from runtime operation identities. The full accepted-frontier count is retained before its first item intent; runtime invocation, created-run and external-attempt charges remain separate facts.

The scalar policy maps returned observations 0/1/2 to zero/one/two ordered children, using the finite integer identity scheme in the application contract. The historical paired controlled cases used the same request and retained artifacts: changing only gamma's returned observation from 2 to 0 changed frontiers `[1,2,3] / [7,10,11] / [31]` into `[1,2,3] / [7]`. Direct and wrapped deployment use the identical adaptive artifact at maximum leaf depths 2 and 3. Additional rounds do not increase invocation depth. Request round/work limits persist under the shared root envelope of 32 accepted invocations, 64 created runs and 96 external reservations, with physical capacity 1 or 2; the fixed root run is separate provisioning cost.

Protocol success does not mean exhaustive application completion. Work- or round-limit stops return `complete=false` with the entire unperformed frontier; only exhaustion returns `complete=true`. A required unknown outcome blocks the round barrier and remains spent, even if a sibling has a known failure. Neither an unknown outcome nor an absent response is converted into observation zero or permission for replacement work. Root closure may prevent any application reply. The effectful profile directly reuses the existing two-member evidence helper, accepts questions of at most 512 UTF-8 bytes, and consumes only its returned disposition for branching; complete answers and citations remain inspectable in the original retained outcomes. It establishes no answer-quality or semantic-completeness claim.

These are bounded implementation results, not a general expressiveness or recovery proof. The source-bound Task2 report records passing controlled direct/nested and paired-result cases and byte/profile checks at manifest `fd7ab336...`; those results predate the rebase onto `56892cb...` and do not qualify the rebased source. Process recovery, unknown/closure schedules, isolated replay, independent faulty-application rejection, final suites and review are qualified separately by source-tagged records under `docs/foundational/phase1h`; only records matching the relevant source establish that source's tested status. Replay checks retained protocol consistency; the independent expected-work ledger checks whether the application follows the intended frontier policy.

3 Pillar II: Structural and Mathematical Operational Semantics

3.1 Small-Step Operational Semantics of Pure Local Reduction (δ_v)

Let $x = (a, g) \in \mathsf{RunID}$ be a logical execution identity, $n \in \mathbb{N}^+$ the next commit sequence, and $\sigma = \langle pc, \rho, status, \kappa \rangle$ an open projection ($status = \text{"}\mathsf{open}\text{"}$). The pure single-step reducer δ_v (`executor.Reduce` in `internal/executor/program.go`) defines the judgment $P, x, n, \mathcal{L} \vdash \langle \sigma, o_{\text{opt}} \rangle \xrightarrow{\delta_v} \langle \sigma', \vec{\iota} \rangle$:

$$\textbf{(R-Receive)} \quad \frac{\begin{array}{c} I_{pc} = \mathsf{Receive}(\mathcal{M}, \beta) \quad o_{\text{opt}} = \mathsf{Some}(o) \quad o.Run = x \quad \|o\|_{\text{json}} \le \mathcal{L}.MaxObs \\ \langle o.Kind, o.Prov, o.Src, o.SrcGen, o.Obl, o.Op \rangle = \\ \langle \mathcal{M}.kind, \mathcal{M}.prov, [\![\mathcal{M}.e_{\text{src}}]\!]_\rho, [\![\mathcal{M}.e_{\text{gen}}]\!]_\rho, [\![\mathcal{M}.e_{\text{obl}}]\!]_\rho, [\![\mathcal{M}.e_{\text{op}}]\!]_\rho \rangle \\ \mathsf{schemaValid}_P(pc, o.Payload) \quad \rho' = \mathsf{bindPayload}(\rho, \beta, o.Payload) \end{array}}{P, x, n, \mathcal{L} \vdash \langle \sigma, \mathsf{Some}(o) \rangle \xrightarrow{\delta_v} \langle \mathsf{clean}_P(\langle pc + 1, \rho', \text{"}\mathsf{open}\text{"}, \kappa \rangle), [] \rangle}$$

$$\textbf{(R-Emit)} \quad \frac{\begin{array}{c} I_{pc} = \mathsf{Emit}(dst_{\text{opt}}, \mathcal{E}, \Upsilon) \quad o_{\text{opt}} = \mathsf{None} \quad id = \mathtt{json}([a, g, n, 0]) \\ \iota = \langle ID : id,\ Run : x,\ Seq : n,\ Ord : 0,\ Kind : \mathcal{E}.kind, \\ Target : [\![\mathcal{E}.e_{\text{target}}]\!]_\rho,\ Gen : [\![\mathcal{E}.e_{\text{gen}}]\!]_\rho,\ Obl : [\![\mathcal{E}.e_{\text{obl}}]\!]_\rho,\ Payload : [\![\Pi]\!]_\rho \rangle \\ \mathsf{authorIntentValid}_P(\iota) \quad \rho_1 = \rho[\forall (k \mapsto e_u) \in \Upsilon : k \mapsto [\![e_u]\!]_\rho] \\ \rho_2 = \rho_1[dst_{\text{opt}} \mapsto \mathsf{Str}(id)] \quad \|[\iota]\|_{\text{json}} \le \mathcal{L}.MaxOut \end{array}}{P, x, n, \mathcal{L} \vdash \langle \sigma, \mathsf{None} \rangle \xrightarrow{\delta_v} \langle \mathsf{clean}_P(\langle pc + 1, \rho_2, \text{"}\mathsf{open}\text{"}, \kappa \rangle), [\iota] \rangle}$$

Lemma 3.1 (Purity and Strong Determinism of δ_v)**.** *For any valid program P, run identity $x = (a, g)$, sequence $n \in \mathbb{N}^+$, limits $\mathcal{L}$, open projection σ, and optional observation o_{opt}, $\xrightarrow{\delta_v}$ is a partial function producing unique canonical JSON bytes for σ' and $\vec{\iota}$.*

3.2 Global Transition Rules at the Runtime Boundary

Embedding δ_v into a global configuration Γ yields the atomic commit and authorization transitions:

$$\textbf{(G-Commit)} \quad \frac{\begin{array}{c} p = \langle x = (a, g), e, n + 1, o_{\text{opt}}, \sigma', \vec{\iota} \rangle \in \Gamma.\mathsf{Prop} \quad \Gamma.\mathsf{Subj}[a] = \langle g_{\text{curr}}, owner, w, retired \rangle \\ g = g_{\text{curr}} \quad \neg retired \quad \Gamma.\mathsf{Runs}[x] = \langle P_v, e_{\text{curr}}, n_{\text{curr}}, \sigma \rangle \\ e = e_{\text{curr}} \quad n = n_{\text{curr}} \quad \sigma.status = \text{"}\mathsf{open}\text{"} \\ (o_{\text{opt}} = \mathsf{Some}(o) \implies o \in \Gamma.\mathsf{Obs}[x] \wedge o.consumed = \bot) \quad \neg\mathsf{scopeClosed}(\Gamma, x) \end{array}}{\Gamma \xrightarrow{\mathsf{Commit}(x, e, n+1)} \Gamma'}$$

$$\textbf{(G-Authorize)} \quad \frac{\begin{array}{c} \iota \in \Gamma.\mathsf{Int} \quad \iota.ID \notin \mathrm{dom}(\Gamma.\mathsf{Att}) \quad x = \iota.Run = (a, g) \\ \Gamma.\mathsf{Subj}[a] = \langle g_{\text{curr}}, owner, w_{\text{curr}}, retired \rangle \quad \Gamma.\mathsf{Runs}[x] = \langle P_v, e, n, \sigma \rangle \\ allowed \iff (g = g_{\text{curr}} \wedge \neg retired \wedge w_{\text{curr}} = \iota.revision \wedge \sigma.status = \text{"}\mathsf{open}\text{"} \\ \wedge \neg\mathsf{scopeClosed}(\Gamma, x) \wedge \mathsf{grant}(\Gamma.\mathsf{Perm}, \iota)) \end{array}}{\Gamma \xrightarrow{\mathsf{Authorize}(\iota.ID)} \Gamma\big[\mathsf{Att}[\iota.ID] \leftarrow \langle Allowed : allowed,\ Reason : reason \rangle\big]}$$

4 Pillar III: Dynamic Concurrency, Orthogonal Authority, and Compositional Congruence

4.1 The Four-Tier Identity Hierarchy and Orthogonal Authority Graphs

Unlike the π-calculus [10] or classical actor models [7, 1], ALAN separates identity into four tiers: (1) Retained Computation (P_v, σ); (2) Logical Subject $a \in \mathsf{Subj}$; (3) Logical Execution

(Run) $x = (a, g) \in \mathsf{RunID}$; and (4) Physical Activation $(x, e) \in \mathsf{RunID} \times \mathbb{N}^+$, alongside ownership revision $w \in \mathbb{N}^+$.

For static provisioned subjects, $\mathcal{G}_{\text{prov}}$ records creation or provisioning history. In the communication graph $\mathcal{G}_{\text{comm}}$, write $M(p, a, o, w_p)$ for an ordinary message route from p to a on obligation o at the sender's revision w_p. Write $T(a, p, o, w_a)$ for a terminal-notification route from the observed subject a to the observer p at a's revision w_a. In the lifecycle graph $\mathcal{G}_{\text{life}}$, $L(p, a, op, w_a)$ grants p the specified operation on a at the expected target revision.

Theorem 4.1 (Authority non-amplification for static grants — Invariant I3)**.** *Between static subjects on these Store routes, acceptance of a fresh lifecycle request requires its matching L grant. Ordinary message admission requires its matching M route, and terminal notification requires its matching T route. Provenance supplies none of these permissions. Possession of any one grant supplies neither of the others.*

Proof. On first authorization, `AuthorizeIntent` requires the requesting run to be current, open and unretired at the revision recorded with its intent. With source permission, `DecideLifecycle` checks for a lifecycle grant matching requester, target, operation and expected target revision. It also checks the target generation, current revision, retirement and operation eligibility. For an existing static target, a missing lifecycle grant yields `grant_denied`. In particular, a monitor with a terminal route can observe failure and still receive this refusal.

For an ordinary send, `DeliverMessage` obtains source authorization. Admission requires the message grant for the sender, target, sender revision and obligation. When the child closes, the Store freezes the destinations and obligations granted at the child's revision. `DeliverTerminal` requires the requested destination and obligation to appear in that list. □

What the requester can learn. I3 separates authorization checks. The public observations of an ordinary requester also include settlements of its own lifecycle operations. `SettleLifecycle` returns the decision to the original requester run with the operation ID, acceptance flag, reason, successor and disposition. The target snapshots used to decide the request stay outside this program interface. For a fresh, source-authorized `Retry` on a current unretired static target with matching grant, generation, revision and sufficient capacity, a failed run and an abandoned run give the same `retried` reply and successor generation. A cancelled run gives that reply when `RetryCancelled` is enabled. An open run and a successful run both return `not_retryable`. For an existing target the Store checks the grant, generation, revision and retirement in that order before retry eligibility. Once those checks pass, a cancelled run with retries disabled yields `retry_cancelled_disabled`. Thus the reply exposes a predicate of target state even without a terminal-observation grant. On the paired post-End schedules of the supervision experiment, the parent's selected reply and final status differ. These runs refute the inference from independent grants to observational noninterference at that interface. The gates and settlement fields are in `internal/executor/lifecycle.go`, `internal/executor/messages.go` and `internal/executor/store.go`.

4.2 Two-Tier Injective Identity Algebra

To eliminate receipt spoofing, transport domain shadowing, and cross-store operation key collisions, ALAN defines:

$$\mathsf{AdmKey}(d, \tau_{\text{raw}}) \triangleq \texttt{json}\big(\texttt{"admission"},\ d,\ \tau_{\text{raw}}\big), \quad d \in \{\texttt{input}, \texttt{message}, \texttt{terminal}, \texttt{lifecycle}, \texttt{external}, \texttt{adapter}\}, \tag{1}$$

$$\mathsf{ExtKey}(\mathcal{N}_{256}, (a, g), n, k) \triangleq \texttt{json}\big(\texttt{"alan/external/v2"},\ \mathcal{N}_{256},\ a,\ g,\ n,\ k\big), \quad \mathcal{N}_{256} \in \{0,1\}^{256}. \tag{2}$$

4.3 Bounded Template Invocations and Conditional Congruence

Theorem 4.2 (Root Resource Conservation and Waiting-Parent Slot Freedom)**.** *Let r be a composition root with envelope $\mathcal{B}(r) = \langle D_{\max}, I_{\max}, R_{\max}, E_{\max}, C_{phys} \rangle$ (`internal/executor/invocation.go`). Across arbitrary interleavings of local steps, template invocations, external adapter calls, root closures, and `SIGKILL` recoveries: (1) total created runs in r's tree never exceed $R_{\max}$, invocations never exceed $I_{\max}$, effect reservations never exceed $E_{\max}$, and depth never exceeds $D_{\max}$; and (2) a parent run suspended at `AwaitInvocation` occupies zero physical transport slots in C_{phys}.*

Definition 4.3 (Admissible Contexts, Dual Readiness, and Budget-Indexed Bisimulation $\approx^{\Delta}_{I,\mathcal{B}}$)**.** The grammar of admissible composition contexts $\mathcal{C}_{\mathrm{adm}}(I_{\mathrm{in}} \Rightarrow I_{\mathrm{out}})$ is generated by:

$$C[\cdot] \;::=\; [\cdot] \;\mid\; C[\cdot] \parallel_{\mathcal{I}} Q \;\mid\; Q \parallel_{\mathcal{I}} C[\cdot] \;\mid\; \mathsf{hide}\ H\ \mathsf{in}\ C[\cdot] \;\mid\; \mathsf{Invoke}_{T,r}(C[\cdot]), \qquad H \cap \mathcal{E}_{\mathrm{ext}} = \emptyset.$$

Structural admissibility does not itself entitle a context to successful completion. Completion additionally requires contract-based accounting of prior spending, outstanding reservations, remaining candidate/peer/wrapper demand and depth at every reachable prefix. Insufficient capacity remains a valid refusal experiment.

Because mailbox admission (`AdmitMessage`/`DecideLifecycle`) is asynchronous with respect to a run's program counter while observation consumption (`Reduce` at `receive`) is synchronous with the 6-tuple guard $\mathcal{M}$, every configuration Γ exposes two refusal/readiness observables at interface I: *mailbox admission readiness* $\mathsf{AdmReady}_I(\Gamma) \subseteq \Sigma_{\mathrm{in}}$ (which target subjects/generations are open vs. closed/retired) and *continuation consumption readiness* $\mathsf{WaitReady}_I(\Gamma) \subseteq \mathsf{Match}_I$ (which 6-tuple guards $\mathcal{M}$ are awaited by suspended `receive` instructions). External interactions are labeled by open transitions $\mathsf{extAuth}(r_{\mathrm{root}}, g_{\mathrm{root}}, \ell, \Pi, \Delta\mathcal{B}) \in \mathcal{E}^{\mathrm{req}}_{\mathrm{ext}}$ and environment-quantified responses $\mathsf{extSettle}(\iota.ID, v) \in \mathcal{E}^{\mathrm{resp}}_{\mathrm{ext}}$ $(\forall v \in \mathsf{Val})$.

A symmetric relation $\mathcal{R}$ is a *Budget-Indexed, Interface-Scoped Failure/Divergence Bisimulation* ($\approx^{\Delta}_{I,\mathcal{B}}$) if whenever $(\Gamma_1, \Gamma_2) \in \mathcal{R}$:

1. **Weak Transfer & Budget Preservation:** $\Gamma_1 \xrightarrow{\alpha}_H \Gamma_1'$ implies $\Gamma_2 \xRightarrow{\hat{\alpha}}_H \Gamma_2'$ with $(\Gamma_1', \Gamma_2') \in \mathcal{R}$ and equal root resource debits $\Delta\mathcal{B}(\Gamma_1 \to \Gamma_1') = \Delta\mathcal{B}(\Gamma_2 \Rightarrow \Gamma_2')$;
2. **Dual Refusal Preservation at Quiescence:** If Γ_1 is H-quiescent, $\Gamma_2 \xRightarrow{\epsilon}_H \Gamma_2'$ for an H-quiescent Γ_2' with $(\Gamma_1, \Gamma_2') \in \mathcal{R}$, $\Delta\mathcal{B} = \vec{0}$, $\mathsf{AdmReady}_I(\Gamma_1) = \mathsf{AdmReady}_I(\Gamma_2')$, and $\mathsf{WaitReady}_I(\Gamma_1) = \mathsf{WaitReady}_I(\Gamma_2')$;
3. **Divergence Sensitivity (Δ):** $\Gamma_1 \uparrow_H \iff \Gamma_2 \uparrow_H$.

Premise audit. Definition of $\approx^{\Delta}_{I,\mathcal{B}}$ requires symmetric weak transfer with equal debit on every matched step, zero-debit matching quiescence with dual readiness, and divergence sensitivity. It is stronger than schema compatibility or upper-bound contract refinement. Unequal-charge local/helper implementations do not refute a theorem assuming it; equal final charges do not establish it. Parallel reasoning additionally needs relation stability under actual residual-budget, authority and closure interference.

Lemma 4.4 (Nonblocking Turn Structure; Confluence Obligation)**.** *For a finite workload while `Store.Run` continues without cancellation, operational-limit exhaustion or store error, its loop offers completion, internal delivery, external launch and ready-local-reduction opportunities. An unmatched receive does not itself hold a transport slot, and background transport does not hold the store mutex across network I/O.*

Code-level justification and limit. The ready-run scan rotates, but `selectReady` loads and scans observations; no constant-time skip or exact global turn bound is claimed. This supports nonblocking structure, not endpoint fairness or schedule equality. Local determinism and termination do not establish a diamond property for competing admissions, closure/authorization or shared endpoint updates. The former unique-quiescent-state inference is withdrawn; general confluence remains open. Interleaving semantics may preserve genuine nondeterminism.

Theorem 4.5 (Conditional Compositional Congruence; Open General Claim)**.** *The proposed implication is: for every context $C[\cdot] \in \mathcal{C}_{adm}$, $\Gamma_A \approx^{\Delta}_{I_A,\mathcal{B}} \Gamma_{A'}$ implies $C[\Gamma_A] \approx^{\Delta}_{I_C,\mathcal{B}} C[\Gamma_{A'}]$, under the originally stated hypotheses: (H1) private namespace separation, declared-port interaction and non-extrusion; (H2) root-correlated external provenance and equal matched debits; (H3) exact six-field correlation routing; (H4) nonblocking peer progress under explicit scheduling/environment assumptions; and (H5) a valid well-founded rank for relevant hidden interaction cycles.*

Status: unproved over the stated grammar. The label preserves the historical claim, not a certification. The former proof is retained in `docs/foundational/phase1i/evidence/manuscript/original-excerpts.md`. Missing arguments are:

1. **Parallel interference:** equal pre-step residual budgets do not prove that the component relation survives a peer debit, closure or authority change. Readiness, outstanding obligations and shared world-response coupling must also be preserved.

2. **Invocation:** admission/refusal, depth, member creation and charges require a separate induction case. A singleton wrapper costs one invocation, one created run and another depth; rejection creates no child/debit. It is not cost-free parallel/hiding syntax, and child/wrapper outcomes differ.

3. **Hiding/progress:** establish transfer, dual readiness and divergence for the actual hidden channels, without erasing shared charges, meaningful refusals, effects or unresolved obligations. The general loop-ranking obligation of Theorem 2.4 is not repaired here.

4. **Concrete scheduling:** abstract trace-set equality differs from a selected `Store.Run` trace. Pure steps, identity ordering and a volatile cursor lost on reopen may change the chosen interleaving. Fairness does not imply confluence or an identical contention winner.

A sufficient parallel rule for single-step matching. Consider a pure interleaving system (x, q, L) with private candidate state x, private peer state q, and a shared ledger L of balances and original decisions. A move is either a transition of (x, L) or a transition of (q, L). Public operation names are fixed, the two components have disjoint ports, and the two readiness projections of the product are the respective unions of their port projections. Fix the visible and hidden labels. This fragment has no synchronization, shared endpoint, or authority change.

Let R_L be a symmetric relation on candidate states at ledger L with equal port readiness. Require each candidate step to match *one* step with the same label (including τ), debit, and successor ledger L', leaving the successors in $R_{L'}$. Specify a *rely* relation D on ledgers. Require $R_L(x, y)$ and $D(L, L')$ to imply $R_{L'}(x, y)$. The peer must guarantee that each of its ledger updates lies in D. Only private state is separated. The peer and candidate can spend the same remaining allowance [8, 13].

Proof. Relate (x, q, L) to (y, q, L) whenever $R_L(x, y)$. A candidate move has its stipulated match. The peer has identical private state and sees the same successor ledger on both sides. For a peer move, copy its transition from the identical (q, L), then apply stability under D at the new ledger. Symmetry gives both transfer directions. The enabled label classes agree, so quiescence matches immediately with zero debit and both readiness unions agree. An infinite

hidden path is matched one hidden step at a time in either direction. This proves the rule for this product semantics. Extending it to weak matches requires controlling the cuts at which a peer can move *inside* a matching path. Exact debits at its endpoints do not settle that question.

Two refusals separated by a peer debit. Fix an open root, stable grants, adequate depth and byte limits, and no external effects. Candidates X, Y each invoke the same two-member template twice with distinct original handles. Only Y sends a public marker after selecting `created_run_limit` followed by `invocation_limit`. An isolated first refusal spends nothing, so the second call sees the same guard. With one invocation and one created run remaining, the peer's singleton can intervene as follows.

original admission	decision	remaining (I, R)
candidate, first pair	`created_run_limit`	$(1, 1)$
peer, singleton	accepted	$(0, 0)$
candidate, second pair	`invocation_limit`	$(0, 0)$

For a two-member call write p_I for $I < 1$ and p_R for $R < 2$. At $(1, 1)$ only p_R holds. At $(0, 0)$ both hold, and the Store tests p_I first. Reversing the guard order would give `created_run_limit` twice and keep Y silent. The guard order is in `internal/executor/invocation.go`.

With the Store's selected reasons, the finite admission/selection model checks two-sided weak transfer with equal per-transfer debits, quiescent admission and continuation readiness on the marker port, and hidden divergence. All 15 isolated starting allowances ($0 \le I \le 2$, $0 \le R \le 4$) relate X and Y. With the peer present, exactly $(1, 1)$ separates them among 24 allowances ($0 \le I \le 3$, $0 \le R \le 5$). Allowing one funded peer debit at any cut in the local model also separates them there. The model exposes original decisions and selections as labels and abstracts successful two-member service to a common response. Its readiness projection distinguishes the root's open marker mailbox from its current receive guard.

Six manually stepped Store executions used three static runs, depth two, and physical capacity one. Both candidates selected the original refusals through their saved handles. The root consumed one marker only for Y with the singleton admitted between the calls. Moving the singleton before or after both calls gave zero markers for either candidate. Looking up the first handle after spending preserved its original refusal. These prefixes stop with the singleton admitted. They witness the effect of an intermediate debit on a parent-visible decision. The general weak congruence and correspondence with automatic scheduling remain open. The model and Store witnesses are in `research/substitution/peer_spending_test.go` and `internal/substitutiontest/peer_spending_test.go`.

The finite model also varies the reasons delivered to the candidate at $(1, 1)$. Reporting all violated allowance predicates gives $\{p_R\}$ followed by $\{p_I, p_R\}$ on the displayed schedule. The original string tests miss both sets. A client testing for that sequence of sets still emits a marker only when the singleton intervenes. Reporting `insufficient_allowance` for either refusal gives the same two reasons with the singleton before, between, or after the calls. Those reasons cannot reveal its placement. A peer can still change acceptance itself. At $(1, 2)$ a pair fits, while a prior singleton leaves $(0, 1)$. Erasing diagnostic fields after the candidate has run leaves any emitted marker visible. The two refusals above belong to distinct invocation handles. The analogous choice in the effect evidence concerns the selected denial for one original mutation.

Bounded behavioral contracts (Phase 1I). The frozen K1 contract separates schema compatibility, asymmetric local refinement and symmetric equivalence. Independently authored local and one-helper candidates preserve a request id and compute a Boolean complement, with explicit failure, missing-reply and pending modes. Five named templates (bare, sequential, parallel, hidden and one wrapper), finite payloads and residual allowances define 3,840

configurations per reference/candidate graph and 84 contract-fixed completion-entitled answer configurations. The reference $C[K1]$, local component relation and candidate products are checked separately. Artifact correspondence inspection, finite model checking, sampled runtime behavior and replay are distinct evidence categories.

Bounded negative result: root success fences a peer. An executed independent model extension reflecting `compositionScopeClosed` (`invocation.go:165--177`) exposes a missing assumption in the frozen parallel reference product. At request id 1/value 0, answer mode, invocation/run limits 3 and depth 2, a correct candidate result lets the root caller succeed before peer S admission; S is then refused with `scope_closed`. Initial completion entitlement is retained and no external `close_root` event occurs. This happens for K1 and both candidate graphs. It falsifies the frozen parallel completion premise/runtime correspondence, not an implication requiring a valid $C[K1]$ premise or the stronger exact-debit congruence premise. The source-specific boundary-extension census records all eight initially entitled parallel configurations failing and the other 76 completing in those finite graphs; the failures remain in scope. Earlier positive reference results used optimistic semantics omitting this ordinary-root-terminal guard. This is bounded model evidence; concrete runtime confirmation and final review are separately qualified.

Reference-model corrections preserve their original failures: collapsed helper/wrapper failure consumption and terminal freeze, and later premature payload visibility. Corrected graph results supersede those versions without rewriting their evidence. Preliminary source-specific checks live under `docs/foundational/phase1i/evidence/model`; they are neither a general theorem nor final runtime qualification. Weak-contract resource and equal-charge scheduling probes retain separate premises and traces. Phase1I qualification/review records govern the executed scope. Acyclic witnesses avoid the general loop-ranking obligation; pure own-state recovery does not establish effectful substitution. Replacing code beneath retained continuations is excluded.

Retained joining and report-source lifetime (Phase 1J). The separately frozen J1 experiment preserves A, B, S, the static root/peer topology, the peer's 32-step prefix, and the original resource allowances. Its root consumes the candidate outcome, retains it, and waits for a correlated ordinary report of the peer's actual S outcome before deciding either success or failure. The peer reports every known S outcome and then succeeds locally; this local status means reporting-program completion, not S success or guaranteed report delivery. An independent causal checker connects the report to the peer's original invocation handle and actually consumed observation. A valid-schema premature report can fool the trusting root while violating J1.

Within the eight original parallel answer configurations (two ids, two values, depth caps 2 or 3, invocation and created-run limits 3, zero effects), the independent finite products preserve the checked resource and root-lifetime obligations but admit a new completion counterexample for K1, A and B: S outcome consumption, report commitment, peer local Success, then first report authorization refused as `source_closed`. The report is an ordinary message, so the invocation-reply exemption does not apply; the root remains open at its required receive. Eventual service does not make a refused authorization successful. This is a bounded counterexample under the modeled local/service interleavings, not an observed failure of the unchanged `Store.Run` scheduler and not an impossibility theorem for every public join protocol.

Concrete delayed-peer samples using unchanged `Store.Run` at physical capacity one demonstrate the intended intermediate state: candidate outcome consumed, S still unadmitted, root open at its join receive. They complete with dynamic charges $A = (2, 2, 0)$, $B = (3, 3, 0)$, plus the two fixed runs, while the original C0 failure persists even with larger capacity. Four acknowledged process-kill/fresh-open samples retain the unfinished join or original committed report and complete. These sampled results and their adverse controls are separately quali-

fied in `docs/foundational/phase1j/02-qualification.md`. They support the causal value of retaining the root in those executions; they do not establish all-schedule completion. The next exposed obligation concerns the source authority needed by required ordinary communication until authorization is retained. The general congruence, compiler and effectful-recovery obligations remain open. The structured-loop termination bound was corrected separately in the preceding manuscript revision; J1's finite graph rank neither relies on nor replaces that structural argument.

Required communication and safe termination (Phase 1K). With frozen J1 candidate B, request id 1/value 0, answer mode, invocation/created-run limits 3, depth 2 and zero effects, a supported public driver realizes the modeled report-source race using genuine original instructions and observations. First authorization after peer termination is refused as `source_closed`; the original report receives a refused admission and the root remains pending. In the paired execution, first authorization is retained while the peer is open, the peer then terminates before admission, and the same report is admitted and consumed by the still-open root, allowing aggregate success. Both executions spend three dynamic invocations/created runs plus two fixed runs. These samples and source inspection support H-auth under an admission-open receiver, stable identities/routes/grants and valid payload: later source termination alone does not invalidate an already allowed first authorization. Consumption additionally requires eligible receiver service. Dispatch acknowledgment can record refused admission and supplies no proof of consumption.

The unchanged automatic runner is qualified separately. At the recorded fresh and preauthorization-restart boundaries, the report is the sole pending internal delivery, peer is open just after Send and root is open. The runner services one internal delivery before its next local commit, excluding the late-first-authorization ordering from that exact boundary. Both samples complete. This conditional source/record result supplies no general drain-all barrier or arbitrary-context scheduler guarantee. Three acknowledged SIGKILL/fresh-Open samples preserve the original database and identities: committed report without authorization; allowed authorization with terminal peer before admission; and refused authorization plus refused admission and dispatcher acknowledgment before retry. The allowed disposition survives reopening and source closure; reattempting the same refused authorization/delivery identities preserves the refusal and pending state. Activation renewal does not replace logical work or refund charges. These are single-host process-crash samples, not a distributed recovery theorem.

The single frozen J2 candidate has the peer wait for a correlated ordinary ACK and lets the root commit that ACK before its final decision. Its independent K1 local/helper product at request id 1/value 0 in answer mode, under the same resource limits, contains 1,408 states and 2,886 edges, with two complete maximal states and two ACK-refusal pending maximal states. The adverse order is report consumption, ACK commitment, root local Success, then first ACK authorization refused while the peer waits. Root local success is distinct from aggregate completion. This bounded model result falsifies the candidate's completion claim under the declared interleavings; no extra handshake round was added.

The exact authored J2 peer fails compilation: its received ACK handle has kind `string`, while its retained S handle has kind `request:invocation:peer`, and the required equality comparison rejects those different kinds. Both roots and the original A/B/S artifacts support partial correspondence checks, but no complete J2 peer artifact, deployment or concrete runtime trace exists. A collection conversion would change the frozen prefix and was not implemented. The failed construction and intended policy tests remain evidence, rather than being replaced by a weakened correlation check. This is a specific authoring obstruction, not a universal non-expressibility result.

Current `author.Send` exposes no retained authorization/admission result, and ordinary

`Receive` does not express a terminal wait. A separate external consumer nevertheless provisions a raw `alan/1d/v1` receiver through the public runtime, delivers an authentic observation from a genuinely terminated peer along its frozen route, and verifies actual selected consumption. Thus the constructor and raw-runtime surfaces differ. If a separately qualified peer program could reach successful End only after consuming the correlated ACK, a matching authentic terminal observation with outcome `succeeded` could supply a stable fact about that consumption; the probe alone does not establish this complete protocol. Invoking the peer would instead change topology/accounting and use invocation-outcome settlement. Exact evidence, rejected oracle histories, source identities and remaining correspondence obligations are recorded in `docs/foundational/phase1k/02-qualification.md`.

Executable substitution under invocation ownership (Phase 1L). A different context C_{inv} uses one public parent definition parameterized by the unchanged A or B candidate. It commits candidate and original S invocations before its first await, then consumes both original outcomes before any normal decision. Status and presence guard payload reads; success requires the candidate's preserved id and Boolean complement, S's original id and zero result, parent/aggregate success and resource compliance. Bound parent artifacts differ at three candidate-binding leaves and their catalogues differ; the builder, guards, result policy and allowance agree. This changes the static-peer context rather than repairing its I/J/K counterexamples. Both candidates receive invocation/created-run limits 3, depth 2 and zero effects, with one fixed parent. A spends two dynamic invocations/created runs and B spends three; this is common-envelope guarantee preservation, not exact-debit equivalence.

An independently derived finite reference checks $C_{\text{inv}}[K1]$ first, preserving K1's local/helper choices, then both replacements over four answer inputs (id in $\{1, 2\}$, value in $\{0, 1\}$). The reference/A/B products contain respectively 1,952/926/1,706 states, 4,276/2,006/3,678 edges and 8/4/4 terminal maximal states; every maximum satisfies the required guarantee, with no recorded failure or cutoff. Every edge is checked before deduplication, and an intrinsic rank under explicit eventual-service assumptions bounds strict progress; maxima count states, not all interleaving traces. Checked finite artifact correspondence and current-source local refinement support this bounded implication, not a general compiler or congruence theorem. Separately, eight automatic concrete samples at physical capacity one complete with actual original outcome consumption; selected public B executions expose both child completion orders while parent consumption remains candidate then S. Atomic runtime settlement is expanded into ordered abstract publication/delivery events, so the model's split intermediate states are not automatically real checkpoints.

Focused controls preserve truthful wrong-id failure while rejecting component conformance and the guarantee, retain actual insufficient-budget refusal, and distinguish failure, absence and pending outcomes. Preserved reference-checker and raw-validator counterexamples required correction and independent rereview; they are evidence-validation defects, not observed runtime substitution failures. One acknowledged B process-crash predicate per normal/race profile has both invocations committed, the candidate outcome consumed once, and admitted S still at its unconsumed request with no produced outcome. Actual SIGKILL and fresh-process opening of the same database complete original work without duplicate consumption or refunded spending. These are single-host samples under stable identities/grants, no cancellation, sufficient operational bounds and eventual eligible service; arbitrary context grammars, universal scheduling, general compiler correctness and effectful/distributed recovery remain open. Exact scope, identities, decisive evidence and review dispositions are in `docs/foundational/phase1l/02-qualification.md`.

Consuming order and reason policies (Phase 1M). Two further alternative parents retain one owner and the original A/B/helper/S components. Both commit candidate then

S and consume S then candidate before any normal decision. C_{se} retains the strict answer policy and guarantee G_{answer}; C_{reason} classifies actual consumed status, reason and presence under G_{outcome}. After validating original S as succeeded/`member_succeeded`, present, with original id/zero, correct candidate success selects `answer`, failed/absent `member_failed` selects `candidate_failed`, and failed/absent `missing_reply` selects `candidate_missing_reply`; other completed evidence selects defensive `invalid_outcome`. Presence guards prevent absent-payload reads. The absent child's reason stays `missing_reply` while the deliberately failing parent/root has `member_failed`. No requested-mode branch replaces observed evidence, and errors still require both joins.

The independently frozen K1-first models retain both local/helper choices. Under common invocation/created-run limits 3, depth 2, zero effects and one fixed parent, C_{se} checks four answer inputs per product (id in $\{1,2\}$, value in $\{0,1\}$): reference/A/B have 1,996/944/1,784 states, 4,376/2,046/3,842 edges and 8/4/4 terminal maxima. C_{reason} adds fail/absent/pending, sixteen configurations per product: 6,072/2,680/6,460 states, 12,880/5,414/13,636 edges, 24/12/12 terminal and 8/4/4 intended pending maxima. Every maximum satisfies its selected guarantee; the reason model's twelve answer configurations separately satisfy G_{answer}. All-mode local refinement and separate open-progress checks support replacement conformance. Every incoming edge is checked before deduplication without failure pruning or cutoff; rank and eventual eligible service qualify progress. Dynamic spending remains two or three under the same allowance, not exact-debit equivalence.

Separately, each context has eight automatic concrete answer samples at capacity one. Public S-first schedules expose both original publication orders and an open parent with S consumed while candidate work is still required, either unproduced or retained-unconsumed. Reason samples authenticate selected envelopes, actual committed classification/branch, terminal execution and the designated unmatched pending receive; pending has no candidate outcome or completed classification. A classification Set and selected decision share one committed step, expanded into two ordered abstract events; executing End occurs later. No durable cut between classification and decision follows. Truthful wrong-id and observation-versus-request controls reject component/profile conformance without rewriting evidence. Finite artifact correspondence and operational bounds supplement these single-host samples under stable identities/authority, valid bindings, no cancellation and eventual service. General congruence, arbitrary contexts, universal compiler/scheduler correctness and new effectful/distributed recovery remain open; Phase 1L's recovery retains its original scope. Exact identities, controls and review dispositions are in `docs/foundational/phase1m/02-qualification.md`.

Callable owning joins and consuming callers (Phase 1N). A new ordinary public join $J[X]$ owns the unchanged candidate and S invocations, commits candidate then S, and consumes S then candidate before normal classification. The sole fixed root O invokes J and consumes that original public outcome. J replies with a bounded record containing the original application id, classification and zero-or-one original answer records; O guards presence, correlation, cardinality and status/reason/tag consistency before committing its decision. Candidate `failed/missing_reply` becomes a present `candidate_missing_reply` diagnostic on J's deliberately failed `failed/member_failed` outcome. Candidate failure has its distinct `candidate_failed` diagnostic. Missing J reports are not diagnoses: Success without a required reply publishes `failed/missing_reply`, whereas Failure without a reply retains `failed/member_failed`; O rejects either absent response as `invalid_join_outcome`. Neither classifier substitutes requested mode for selected evidence.

The common allowance is four invocations/created runs, depth three, zero effects and one fixed root. J's prior admission costs $(1,1,0)$, leaving residual allowance three; candidate/S enter at depth two and B's original helper at three. Only accepted original S admission maps to the local environment's arbitrary-time debit. A spends $(3,3,0)$ and B $(4,4,0)$; this preserves a

common envelope, not exact-debit equivalence. No outer sibling adds another interfering debit.

A reviewed restricted conditional constructor argument (ARG-1) supplies $J[X] \preceq J[K1]$ for candidates in the declared e/helper event semantics, given a relation preserving original identity/protocol/outcome, source closure, ledger, depth and future-demand facts; matching single events or public-field-preserving private stutters; bilateral environment/caller readiness; equal authorization sidecars; and separate all-mode progress/nondivergence. A weak matching-path variant additionally requires stable intermediate guards and no nonmatching protocol events. The public environment supplies stable bindings/authority, an open eligible caller, sufficient finite limits, no cancellation/retirement or pending release, and eventual eligible service. The independent $J[K1] \models K_{\text{join}}$ premise then gives contract preservation; adding fixed O control yields G_{outer}. This is a source-level conditional argument, not a machine-checked generic theorem. At Phase 1N's close, ARG-2 remained open: arbitrary weak refinement of unchanged K1 had no proved normalization/extension to the stronger intermediate-guard, caller-readiness and authorization-sidecar premises. Neither the original exact-debit definition nor the open general congruence claim changes.

The independently frozen K1-first reference and A/B products cover sixteen inputs each (id in $\{1, 2\}$, value in $\{0, 1\}$ and answer/fail/absent/pending), retaining K1's local/helper alternatives in every mode. Their respective state counts are 6,376/2,864/6,644, edge counts 13,232/5,622/13,844, terminal maxima 24/12/12 and designated pending maxima 8/4/4. All recorded property/rank failures, refusals, stuck states, cutoffs and cycles are zero. Separate aligned local relations and open-progress checks support the finite lifted A/B instances of ARG-1; they do not prove ARG-2. K_{join} and G_{outer} project the same products, not two independent censuses. Pending maxima retain the actual unmatched candidate receive, completed original S consumption, open J awaiting e and open O awaiting J, without a diagnostic or terminal publication. Every incoming edge is checked before deduplication; zero successors establish finite maximality. Finite artifact/guard correspondence and operational bounds are separate evidence.

Concrete normal/race profiles each include eight automatic answer samples, six representative A/B fail/absent/pending samples, one supported late-reply execution and four genuine controls at physical capacity one. In the supported B/absent execution, J commits its diagnostic Send, executes Failure, and only then obtains its first reply authorization and admission; its original failed outcome retains the diagnostic, which O actually selects before committing the matching failure. The permitted closed-source exception is the original entry-member reply bound to the actual caller/generation/obligation with unchanged revision, template and schema checks; invocation authorization receives no such exception. No host-fabricated observation or authoritative-state edit supplies this schedule. The additive raw mapping and independent audit authenticate original emitted handles, selected histories, actual instructions/cursors, report transport and outer decisions. Classification plus ghost decision share one Set commit; End/freeze and publication/delivery have explicit atomic groups, so abstract intermediate states are not automatically durable cuts.

Genuine no-report controls make O reject absent J evidence; a genuine false report passes provenance and O's faithful public policy while failing constructor truth; a deliberate-failure component under an absent request preserves truthful classification while failing K1 mode conformance. These are distinct failed premises, not conforming substitution counterexamples. Detached decoder attacks and incomplete-prefix claims are separately rejected. Shared protocol formats, historical machinery and runtime/replay dependencies limit independence; pure replay checks retained consistency. Runtime review and final manuscript qualification are tracked in `docs/foundational/phase1n/06-review.md`; exact evidence and scope are in `docs/foundational/phase1n/03-qualification.md`. Arbitrary contexts/candidates without the supplied relation, recursive tree closure, universal compiler/scheduler correctness, exact-debit equivalence and effectful/distributed recovery remain open; historical recovery retains its

original predicates and source identities.

Weak refinement and original-state accounting (Phase 1O). The reviewed source argument distinguishes the stronger P_{stage} projection, which already compares original-state choice/ticket/demand stages, from P_{boundary}. Within A_{op} with fixed operational owner framing, and under the separate progress and E_N premises, the former retains its weak relation and derives the remaining ARG-1 bridge duties. For the latter, unrestricted ARG-2 remains open under both event-visible F_{event} and fact-visible F_{fact} readings. For an original candidate $X \in A_{\text{op}}$ with fixed operational owner framing, an additional $\mathsf{Account}_N(X; \alpha, \lambda)$ certificate, together with a supplied W_N^σ weak relation and separate $\mathsf{Progress}_N$, yields a suitable relation $R_g \subseteq \mathsf{States}(X) \times \mathsf{States}(K1)$ under E_N. The fixed maps α on original states and λ on original edges check initial tickets, faithful accounting, ordered updates, branch/mode legality and every original edge/atomic group. They do not assume reference-demand equality, ARG-1 readiness or progress. This is a sufficient representation condition; neither its necessity nor its existence for every boundary-projection candidate is established.

The preserved R1 rejection identifies the missing inference: lawful history accounts can assign possible and excluded helper demand to the same original state. Operational transfer through a history extension does not alone make demand an observation on that state. Preserving every accounted demand through a projection requires and suffices for demand to be constant on its reachable fibers; the full state-accounting record above is a stronger sufficient condition. The corrected argument does not silently add history to X's state domain. The merge is a diagnostic of the failed representation argument, not a conforming candidate counterexample, a nonexistence result or a demonstrated specification gap.

Fixed reference choices preserve current constructor/protocol guards; this differs from endpoint demand equality. Fixed owner framing and mandatory facts supply immediate caller matches, while eventual finite original S service opens J's selection gate. Given the coarse relation, compatible legal first-authorization histories admit the retained-record extension, preserving each run's own raw provenance. These derived portions do not supply E_N's stable identity/authority/bindings, open eligible callers, no closure/retirement/release, eventual eligible service or adequate operational limits. J's prior debit remains $(1, 1, 0)$ within full I=R4, and original S remains the sole q debit. Progress remains separate from safety. Full-domain W, Progress and Account, when supplied, restrict configuration-wise to the residual profile; a profile certificate does not qualify the full contract. Conditional ARG-1 lifting still requires the independent $J[K1]$ reference/product premises for K_{join} and G_{outer} preservation.

This is a source-reviewed argument, not a machine-checked theorem. Corrected finite certificate checks address only declared machinery, accounting, normalization, demand/current-guard and legal-history clauses. In the declared $F_{\text{fact}}/P_{\text{boundary}}$ fragment, both supplied weak relations pass; the premature R_w has eight endpoint-demand inequalities and normalized R_g has no named strong-clause failure. These are supplied-witness results, not whole-candidate conformance, runtime execution or existence search. The corrected argument, preserved rejection and scope boundaries are recorded in `docs/foundational/phase1o/02-arg2.md`, `docs/foundational/phase1o/03-finite-contract.md` and `docs/foundational/phase1o/05-qualification.md`. Arbitrary-context congruence, universal compiler/runtime correctness, recovery and exact-debit equivalence gain no new theorem.

Direct preservation by the callable join (Phase 1P). Fix $\sigma = (P_{\text{boundary}}, F)$ with $F \in \{F_{\text{event}}, F_{\text{fact}}\}$. Every original finite $X \in A_{\text{op}}$ with an original-state relation R_w satisfying $W_N^\sigma(X, R_w)$ and separate $\mathsf{Progress}_N(X)$, both on all sixteen inputs of D_N, satisfies $J[X] \models K_{\text{join}}$ and $O[J[X]] \models G_{\text{outer}}$. This source-level theorem uses the independent reference guarantee $J[K1] \models K_{\text{join}}$, fixed O policy and original E_N assumptions, including stable bindings and authority, open eligible callers, no extra input or release, adequate finite limits and eventual

service of continuously eligible work.

J's accepted admission charges one invocation and one created run, leaving three of each. Every remaining original invocation (e, optional h, and S) can spend at most once. Unadmitted slots therefore bound remaining lawful demand on each history, even when histories that allow or exclude h reach the same candidate state. Permission still requires the original branch and ticket laws, an actual enabled commitment and the kernel's authorization and admission guards. No equality of exact future demand is needed. ARG-2, which asks for an original-state relation satisfying ARG-1, remains open under both boundary readings.

Projection retains every private step of X and counts S admission as the sole q debit. S completes independently of e and opens J's gate for selecting a retained e outcome. At each gate the next prerequisite stays eligible until serviced, so an erased suffix cannot hide enabled local work. Local progress then gives the exact mode boundary. In completed modes J classifies the selected original S and e outcomes. O selects J's original outcome, whose present report carries the singleton $(\mathrm{id}, 1 - \mathrm{value})$ on success, or zero answers and the distinct diagnosis of candidate failure or missing reply on failure. O commits the corresponding decision before root publication. In pending, after consuming its input and any chosen helper result, e waits at its unmatched receive, J has consumed S and awaits e, and O awaits J without a decision or publication. The argument is in `docs/foundational/phase1p/02-preservation.md`.

Transport across diagnostic relays (Phase 1Q). It remains open whether closure at positive relay depth holds for every original finite $X \in A_{\mathrm{op}}$ satisfying either boundary reading of W_N^σ with an original witness and separate $\mathsf{Progress}_N$ on all sixteen requests. At Phase 1P's original binding and initialization, completed publication follows J's selections of S and e, consumption of any chosen helper outcome by e, and settlement of J's bound reply. This yields $K_{\mathrm{report}}(4,3)$ in addition to K_{join}. Even on failure, all owned descendants and captured replies are settled by first publication.

A relay whose sole child satisfies $K_{\mathrm{report}}(b,h)$ at each induced binding satisfies $K_{\mathrm{report}}(b+1, h+1)$. Here b bounds both invocations and created runs including entry, and h bounds depth from the caller. When the child publishes, the relay selects its outcome on the original handle and returns the flat report through its own bound reply. Both failure diagnoses remain present and distinct. In pending, the relay waits on its child without publishing. Given the base export at each actual binding, induction gives $K_{\mathrm{report}}(k+4, k+3)$ for k relays and G_{outer} for the fixed caller under the translated environment.

The original premise does not specify how private instructions interpret a rebound operation or preserve choices, successors and continuous eligibility across bindings. In two public executions at the same depth, the same artifact selected its helper outcome and compared the saved handle string with one fixed literal. Changing the root subject reversed that comparison. A counterexample still requires a conforming original candidate and a legal relocated execution that violates the contract. See `docs/foundational/phase1q/04-qualification.md`.

Certifying opaque use of invocation handles (Phase 1R). The opt-in checker assigns each register one lifetime sort. Ordinary data retain their literal values. An invocation supplies a handle indexed by its declaration. Handles may be copied, compared for equality within their sort, or used to await the matching original operation. The template check covers the exact lowered programs, initializers and declarations of the candidate and its owned singleton helpers. Following the use of permutations for names in nominal logic [14], the proof pairs fresh commitment occurrences by a growing partial bijection. Injectivity preserves both alias equality and the inequality of distinct commitments. Induction on the checked instructions preserves and reflects each private step and its eligibility, including deletion of absent bindings and exhaustion of a loop.

For either boundary reading, assume the original W_N^σ witness and separate $\mathsf{Progress}_N$ on

all sixteen requests. The candidate they qualify must be the certified machine at its original binding, or have an independently established correspondence there covering private steps, faults and maximality. Under the preceding reference, policy, legal binding and continuous-eligibility service premises, with adequate representation and operational bounds in both frames, the argument supplies the base export at each induced binding. Every finite legal $T_k[X]$ then satisfies $K_{\mathrm{report}}(k+4, k+3)$ and $O[T_k[X]]$ satisfies G_{outer}. The implication for unrestricted A_{op} remains open.

In eight public executions at one fixed request id and value, J invoked the same certified B directly below O and with one relay between O and J. In both placements B invoked and awaited the same helper. Both placements produced the correct answer, distinguished member failure from a successful End with a missing reply, and retained the unpublished pending wait. The legacy spelling probe still took opposite branches and failed the new check at its first handle-to-string append. A different certified candidate returned the wrong answer. These executions qualify the measured instances. The conditional theorem rests on the source argument. See `docs/foundational/phase1r/03-qualification.md`.

Original conformance of retained B (Phase 1S). Place the unchanged B and its helper in the original local caller and protocol environment D_N. On all sixteen requests, the supplied F_{fact} witness relates 2,240 pairs of states. B consumes its input and then commits its helper invocation. K1 can match that commitment by first choosing the helper silently. Under F_{event} the choice is visible, and the same prefix has no matching event in the retained instructions. In pending mode B enters its unmatched second Receive by a branch. No second consumption occurs.

A separate open-progress check covers 784 states and 1,184 edges, with one serviced maximum per request. Every maximum includes the original helper selection and the concurrent debit. The exact private instruction correspondence and the fact-reading witness therefore discharge B's original behavioral premise in Phase 1R. For every finite legal k, $T_k[B]$ satisfies $K_{\mathrm{report}}(k{+}4, k{+}3)$ for its original leaf, and $O[T_k[B]]$ satisfies G_{outer}, under the same independent reference, policy, open-frame, service and per-placement adequacy assumptions.

The closure-inclusive local kernel and the concrete Store differ on a bound reply. In the original J placement the helper committed its correct reply to B, the root was cancelled, and the helper ended. The Store then authorized and accepted that reply. At the same cut the local kernel requires refusal because the scope is closed. This execution refutes exact correspondence across arbitrary Store cancellation schedules. The positive relay result retains its open-frame hypothesis. See `docs/foundational/phase1s/04-qualification.md`.

Finite owning trees (Phase 1T). Every finite legal tree built from the original callable join with A or B at each leaf, a unary owning relay, and a binary owning fork preserves the following guarantee. Each leaf independently receives one of the sixteen original requests in D_N. All eventual outcomes below assume service of continuously eligible work and a common sufficient allowance for invocations, created runs and depth. The fork commits both invocations before awaiting the left and then the right by their original handles. Equal application ids can therefore occur on distinct branches. If all leaves answer, the tree succeeds and returns their id-preserving complements in their respective positions. If every leaf completes but some candidate fails or ends without its reply, the tree fails with a present product. Each affected position retains its specific diagnosis and every answering sibling retains its answer. At first publication, all owned descendants and captured replies have settled.

A genuinely pending leaf leaves its ancestors unpublished. A fork blocked on its left child may already hold a delivered, unconsumed outcome from its right. The source argument uses the actual retained leaf machines, the generated parent instructions and the open, pure Store protocol. It requires stable authority and bindings, sufficient representation and operational

limits at each placement, and no external closure, extra work or release of a pending receive. Charge each accepted occurrence once against the finite slots of the whole tree. With the fixed outer root counted separately, a two-leaf fork admits AA, AB, BA and BB under the same nine-slot allowance and depth four. They spend seven, eight, eight and nine slots respectively.

An independent finite model covered all 2,176 declared configurations. The public Store campaign exercised 80 automatic configurations and 14 manual schedules and adverse cases at physical capacity one. A branching cancellation prefix preserved the earlier counterexample. A bound reply first received permission after both root cancellation and sender End. The positive theorem applies to open placements.

5 Pillar IV: Algebraic Effects, Split-Phase Operational Handlers, and the Epistemic Effect Lattice

5.1 Split-Phase ITree Lowering versus the Asynchronous Handler Algebra

At the pure syntax and local reduction level (δ_v), factoring $\mathsf{Vis}(e : E(A), k : A \to X)$ into `emit` and `receive` is a syntactic bimorphism $\mathcal{S} : \mathsf{ITree}_{E,R} \to \mathsf{ITree}_{E_{\text{split}},R}$ over $E_{\text{split}} = \mathsf{Emit} \uplus \mathsf{Receive}$:

$$\mathcal{S}\big(\mathsf{Vis}(e,k)\big) \triangleq \mathsf{Vis}\Big(\mathsf{Emit}(e),\ \lambda id.\ \mathsf{Vis}\big(\mathsf{Receive}(id),\ \lambda a.\ \mathcal{S}(k(a))\big)\Big).$$

However, in Xia et al. [21], `interp` applies a monadic handler $h : E_{\text{split}} \rightsquigarrow \mathsf{StateT}(\mathcal{D}, M)$ synchronously inside the evaluator's thread, which cannot represent effects interrupted across the four distributed crash windows W_1 (post-reduce, pre-commit), W_2 (post-commit, pre-authorize vs. cancel), W_3 (allowed reservation with no durable result), and W_4 (post-receipt, pre-settlement).

Evidence for one original effect. Fix a store namespace, an original operation, and its reservation and receipt contract. We track whether the original mutation was suppressed or had an allowed reservation, and which canonical receipt, if any, establishes its application. Let R be the set of retained denial reasons and V the possible canonical receipt values. Each consistent state has one of four forms.

- $B = \bot_{\text{unauth}}$ carries no decisive reservation evidence. An allowed first source permission followed by no reservation yet still projects to B. The journal retains that permission separately.
- $S_r = \mathsf{Suppressed}(r)$ records the decisive refusal for the original mutation, with selected reason $r \in R$. Examples include `source_retired`, `source_closed`, `grant_denied`, `scope_closed`, and `effect_limit`.
- $U = \mathsf{Unknown}$ carries an allowed reservation. Historical application remains unresolved, even if a later dispatch opportunity is refused.
- $K_v = \mathsf{Known}(v)$ records the correlated receipt $v \in V$. The dispatcher is trusted and the endpoint issues this receipt only after durable application. A failed adapter response establishes its returned observation while application can remain unresolved.

The information order on $\mathbb{O}_{\text{eff}}^{\text{cons}}$ is generated by $B < S_r$ and $B < U < K_v$. Adjoin $T = \top_{\text{conflict}}$ above every element to obtain $\widehat{\mathbb{O}}_{\text{eff}}$. The five constructors give $3 + |R| + |V|$ elements. The lattice is infinite if either index set is infinite. A finite profile can have a finite carrier. For nonempty V its height is three edges. The full join table uses $J_S(r,s) = S_r$ if $r = s$ and T

otherwise, and $J_K(v, w) = K_v$ if $v = w$ and T otherwise.

$\sqcup$	B	S_s	U	K_w	T
B	B	S_s	U	K_w	T
S_r	S_r	$J_S(r, s)$	T	T	T
U	U	T	U	K_w	T
K_v	K_v	T	K_v	$J_K(v, w)$	T
T	T	T	T	T	T

Each entry is the least common upper bound, so join is commutative, associative, and idempotent. Comparable pairs meet at their lesser element. Distinct K_v, K_w meet at U, and all remaining incomparable pairs meet at B. Settlement selects the first recorded result. The evidence about the original operation includes every distinct qualifying receipt the recorder commits, including offers rejected for settlement.

Two denials conflict when they assign different selected reasons to the *same authoritative decision*. Retirement and closure can both hold. At a stable generation and revision the first source decision for a mutating operation selects retirement when both hold. Likewise, $K_v \sqcup K_w = T$ for distinct canonical results even if both offers support the weaker proposition that some application occurred. Decisions for later dispatch opportunities or new retry generations have their own scope.

The two-call invocation witness uses the same distinction between a selected reason and all the predicates that hold. There the reason is a public input to the caller's next branch. Here S_r records what one mutation decision established.

Belnap's information order distinguishes absence from conflict in a four-element diamond alongside a separate truth order [2]. In ALAN, whenever R and V are nonempty, B, U, K_v, S_r, T form the pentagon N_5. Since $U \leq K_v$, modularity would require $U \sqcup (S_r \sqcap K_v) = (U \sqcup S_r) \sqcap K_v$. The left side is U and the right side is K_v. The lattice is therefore nonmodular and hence nondistributive. It is also ungraded. The saturated chains $B < U < K_v < T$ and $B < S_r < T$ have the same endpoints and lengths three and two, so no rank increases by one on every cover. A monotone confidence score would require a separate choice of scale.

First settlement and durable receipt evidence. Fix one allowed original operation on the single-host raw or ordinary mutating-adapter path. Let H be the set of distinct canonical receipts for that original in committed records. A raw receipt must validate against the retained request. An adapter receipt must be a validated success under the original output contract. Receipt identity uses the complete canonical descriptor, so different response metadata can distinguish receipts with the same program-visible payload. Interpreting either as evidence of application assumes a trusted endpoint that issues it only after application. An unsuccessful adapter return can settle the interaction while contributing no receipt to H. The evidence projection is U for empty H, K_v for $H = \{v\}$, and T when H contains at least two values.

The first result fixes what the continuation can consume. When another valid receipt for that original differs from the first result, the recorder stores its complete canonical descriptor under the original operation ID and binds it to the first result by digest before returning the conflict. Reoffering the same receipt adds no row. For a qualifying receipt v, a successful recording or an acknowledged rejection for settlement gives $H' = H \cup \{v\}$. Its projection is therefore the join of all committed receipts above U. When distinct v and w contend for an initially empty result slot, reversing their arrival reverses the settlement winner while both journals project to T. Replay verifies the bindings and reports the evidence for originals with additional receipts. It issues no new request or settlement.

Before this journal extension, the recorder returned the conflict and retained only the first result. With first receipt v, that projection stayed at K_v. It was monotone but omitted the conflict a live caller had observed. Earlier discarded offers cannot be reconstructed. A Store in

format 3, 4 or 5 acquires the additional journal in the same transaction as its first qualifying rejected offer and becomes format 6, 7 or 8 respectively. A crash before that commit preserves the old state. Once it commits, recovery retains the new evidence even if the caller misses the acknowledgment or the first result has already been consumed. Foreign keys, unsuccessful offers and decisions in the separate receipt-read and reconciliation protocols do not supply conflicting receipts under this rule.

Theorem 5.1 (Source closure before first permission)**.** *Fix a committed mutating external or adapter intent of an open source with stable revision and generation. Suppose a granted* `CancelRun` *actually closes that source, or* `RetireSubject` *retires it, before its first source permission decision. Assume all launches of this original operation go through the trusted dispatcher. The first decision denies permission and the dispatcher cannot send the mutation. At first permission the raw reason is* `source_retired` *if the subject is retired, otherwise* `source_closed`*.*

Proof. Lifecycle and first permission serialize. Permission reads the closed projection or retired subject and persists its refusal. A later lookup returns that decision. Each route checks it at reservation, and a denied reservation cannot expose a mutating worker. □

Withdrawn launch clause. The old second clause forbade new physical launches after source cancellation. Phase 2A falsifies it on both baseline routes. A worker acknowledged its pre-call barrier, a granted operator closed its actual source, the harness released the worker, and an independent endpoint received and applied its original request. On that historical baseline an unscoped source also launched after cancellation between first permission and fresh reservation. Fresh member reservations check applicable enclosing closure and effect allowance. Cancelling the tested static member or its enclosing root before reservation denied it without a charge. Closure after reservation left the worker able to launch. A new baseline Run also sent the same raw idempotent key after cancellation. It withholds unresolved non-idempotent raw and allowed adapter reservations. A pre-closure launch that applies later is a distinct ordering. Late recording and settlement preserve the closed continuation. The original proof and ordered histories are archived in `docs/foundational/phase2a`.

Phase 2B local admission and quiescence. Fix one retained Store namespace and either an original source generation or an actual owning scope. Let R be its effective durable revocation, $A(w)$ the current admission of one physical opportunity, $L(w)$ entry to raw HTTP or the registered adapter's Execute, and $H(w)$ the end of local handling of its return or withdrawal. Under the single-host owner lock, the gate reads current eligibility and registers that individual opportunity in the same critical section. A revocation that wins excludes a new admission, including a same-key idempotent resubmission. If $A(w)$ wins, the order $A(w) < R < L(w)$ remains possible on both paths.

Admission requires the original source generation and revision to be current, its subject unretired, its own projection open, and any applicable owning scope eligible. Cancel, abandonment, retirement, supersession, and a source's own successful or failed terminal transition can stop fresh work. A terminal top-level resource root closes its scope. A successful unpublished nested freeze still permits eligible descendants. Non-success freezes, retirement closures and published nested outcomes participate in the ancestor check. A successful source that ends before admission can leave its captured effect outstanding and its aggregate waiting. Refusing an exposure preserves the original grant, request, key, reservation and charge.

The public host wait acknowledges Q only for an already revoked target. Closing a Store drains its owner and releases the flock. An idle Run or owner shutdown cannot certify revocation of an open logical source, which a later eligible owner can serve. Each admitted worker remains covered through queued completion, recording and settlement, including distinct exposures of

the same key. A closed continuation need not select a result. A genuine unresolved return may finish without one. A matching accounting failure is reported to the caller. The wait may settle a surviving recorded result without an active scheduler. If Q succeeds, no covered worker can enter L later: any earlier admission must have finished handling, and a later admission would contradict the durable guard. This conditional argument assumes trusted dispatch and subsequent owners implementing the same policy. It covers both automatic dispatch and the supported synchronous raw route. Waiting releases the Store lock. Obtaining Q can still depend on cooperative transport return, accounting service and local I/O. A deadline can expire first. See `docs/foundational/phase2b/01-contract.md`.

Phase 2C read and import authority. The original interaction and the application's later evidence occupy distinct records. In the schema-5 controlled profile the issuing entry sends its committed operation ID to the live caller, receives an acknowledgment through the invocation's typed handoff, and terminates. The caller has already forwarded the ID to a resolver outside the original owning scope. The issuer's frozen reply records issuance. Its successful aggregate can still be waiting on that one captured result. Possession of the ID grants no authority. A provisioned read grant selects one resolver generation, one original operation and one ledger endpoint. The Store binds the retained authorization, request, key, preparation and exact wire before admitting a read. A separate import permission is needed to reconstruct the missing controlled counter success from the ledger's original receipt.

Each query has its own key and semantic debit. Each physical read spends one of two durable opportunities before entry. At one serialized snapshot the endpoint compares the original wire with the independent application ledger. One exact match supplies the immutable receipt, multiple matching applications yield ambiguity, and absence means only that no application was visible there. Only independent original-packet service changes the business projection. Query audit writes and local debits remain visible costs. A found observation can establish application without changing a previously selected transport failure or the issued reply. See `docs/foundational/phase2c/01-contract.md`.

Phase 2D controlled composition. Fix one static caller P that owns two issuing children L, R and one finite, representable Series 1 tree T built from the original A/B leaves, Relay and Fork. P invokes all three before its first selection. Two distinct, provisioned handoffs carry the actual original operation identities. Each separately rooted resolver has read and import authority for its one original on the same controlled counter ledger. On selecting its first compatible found result it saves the target, mutation number, total, adoption classification and logical query identity. The Store retains the full receipt. A subsequent duplicate or transport error remains a separate query outcome. Conflicting evidence remains visible. The caller selects both original invocation outcomes, the pure outcome, and both authentic reports before deciding. If an original interaction failed, the corresponding classification preserves that failure alongside later evidence of application.

On an initially empty ledger with just these two unique applications, the first receipt has mutation number one and its own delta as total. The other has number two and the sum. Reversing world order exchanges the roles that receive these fields. Equal deltas still identify distinct operations through their bindings. Disjoint invocation occurrences and ports frame the closed pure subtree. Relative to the original Series 1 bounds its two singleton siblings require two additional dynamic invocation and created-run slots, and the parent root needs two mutation debits. A pure diagnosis retains both business outcomes. A pending tree can leave both originals selected, both reports delivered but unconsumed, and the resolvers waiting for acknowledgments. See `docs/foundational/phase2d/02-frame-and-completion.md`.

5.2 Separating Probabilistic Intelligence from Deterministic Execution

In `examples/evidence/evidence.go` and `examples/investigation/investigation.go`, LLM invocations (`evidence-model`) are compiled into split-phase `author.Request` and `author.Await` steps surrounded by pure deterministic guards: budget enforcement (`model_spent < model_limit`), tool-name verification (`plan_tool == "lookup"`), deterministic span-exclusion copying across revision rounds, and byte-exact citation provenance verification across `document`, `version`, `snippet`, `start`, and `end`.

6 Pillar V: Crash-Consistent Operational Semantics and the Two-Regime Fundamental Theorem

6.1 Two-Tier Configurations, the Recovery Operator, and the Window-W_3 Impossibility Theorem

We decompose a single-host ALAN configuration into $\Gamma = \langle \mathcal{D}, \mathcal{V}, \mathcal{W} \rangle$, where $\mathcal{D}$ is the durable SQLite WAL/FULL store, $\mathcal{V}$ is the volatile activation state, and $\mathcal{W}$ is the external world state. Let $\mathsf{Commit}(\langle \mathcal{D}, \mathcal{V}, \mathcal{W} \rangle) = \langle \mathcal{D}, \emptyset_{\mathcal{V}}, \mathcal{W} \rangle$ and $\mathsf{Crash}(\Gamma) \ni \langle \mathcal{D}, \emptyset_{\mathcal{V}}, \mathcal{W}' \rangle$. Upon recovery, $\mathsf{Recover}$ validates ledger integrity and increments activation epochs $e \mapsto e + 1$ for open runs. Define $\mathsf{Unresolved}_{\text{non-idem}}(\mathcal{D})$ by an allowed reservation and no corresponding durable receipt/result, restricted to non-idempotent raw attempts and adapter attempts outside the trusted receipt-read recovery profile. A retained unsuccessful adapter observation leaves this set even when application remains unresolved. A subsequent Run withholds those missing-result attempts and fabricates no observation. Only a live source whose next instruction requires the missing result stays at its receive. Other private steps and other runs may progress. Host reports expose the original operation. A missing schema-5 receipt-read result permits a fresh physical read only with current eligibility and a remaining durable opportunity. A previously allowed raw idempotent reservation with a missing receipt may be considered again. Transmission requires current dispatch admission for the original immutable request. A recorded receipt can instead proceed to settlement.

Proposition 6.1 (Impossibility of Unconditional Symmetric Crash Bisimulation in Non-Idempotent Window W_3)**.** *Let $\iota \in \mathsf{Unresolved}_{non\text{-}idem}(\mathcal{D})$ be an authorized in-flight intent in Window W_3 targeting a non-idempotent endpoint $\mathcal{W}_\ell^{non\text{-}idem}$ without an out-of-band read-only receipt-lookup channel, and let $k(v)$ be a continuation whose observable output depends on v. No deterministic recovery operator $\mathsf{Recover}(\langle \mathcal{D}, \emptyset_{\mathcal{V}}, \mathcal{W}' \rangle)$ can simultaneously guarantee (1) At-Most-Once World Safety ($\#\mathsf{Apply}_\ell \leq 1$ across $\mathcal{W}' \in \{\mathcal{W}, \mathsf{apply}(\mathcal{W}, \alpha)\}$) and (2) Unconditional Symmetric Forward Bisimulation with the uninterrupted execution where the live socket in $\mathcal{V}$ delivers v to $k(v)$.*

Proof. Because $\mathcal{D}$ is identical whether the request dropped before reaching $\mathcal{W}_\ell$ ($\mathcal{W}' = \mathcal{W}$) or mutated $\mathcal{W}_\ell$ just before the response socket died ($\mathcal{W}' = \mathsf{apply}(\mathcal{W}, \alpha)$), re-dispatching α violates (1) in the latter case ($\#\mathsf{Apply}_\ell = 2$), whereas withholding re-dispatch leaves k without v, violating (2) in the realization where the live socket returned v. □

6.2 The Two-Regime Fundamental Theorem of Durable Computation

Let $\approx_{\text{obs}}$ require logical store identity modulo activation epochs and symmetric future $\approx^{\Delta}_{I,\mathcal{B}}$ behavior. Identity and epoch-independent local reduction are necessary evidence, not an automatic proof: shared resource/closure interference, admissible response coupling and concrete scheduling remain obligations even with no unknown non-idempotent attempt.

Directed epistemic prefix/safety simulation $\preccurlyeq_{\text{ep}}$ requires historical identity at a matched committed cut and conservative information, no automatic retransmission of an unknown non-idempotent attempt, and explicit peer/reconciliation obligations. A peer's lack of an await dependency is insufficient for symmetric comparison: resource, authority, closure and shared-world interference must be absent or accounted for by a stable relation. A live source at a receive requiring only the missing result retains its original await absent new evidence or closure. A child whose next instruction is private failure can terminate and its parent can select `member_failed` while the effect is still unresolved. Late evidence may advance $\mathsf{Unknown}$ to $\mathsf{Known}(v)$, but rejoining $\approx_{\text{obs}}$ also requires matched continuations, world, authority and residual resources. Closure may prevent consumption without invalidating retained evidence. Conservative safety alone does not establish the full simulation.

The supervisory public-output reconvergence condition (Markdown Definition 6.5) is application-specific, not a separately checkable substitution proof. This hypothetical context needs both a legitimate observation and explicit lifecycle authority over an eligible ordinary run. A generation Retry is a new business attempt and leaves the original uncertainty intact. Invocation membership refuses Retry with `invocation_retry_unsupported`. Retry debits and root-correlated external actions remain visible. Hiding a private generation or `spent` register cannot supply exact-debit equivalence; "modulo retry debit" would need a separately defined observation relation and is not claimed here.

Theorem 6.2 (Two-Regime Durable Computation; Qualified Claim and Open Simulation Obligations)**.** *For a reachable configuration under a fixed semantic version:*

1. ***Part I (Historical Replay Completeness and Zero-Effect Soundness):*** *read-only* `Replay` *reconstructs historical projections and intents without dispatching effects or mutating the retained store.*

2. ***Part II (Proposed Symmetric Future Relation):*** *when* $\mathsf{Unresolved}_{non\text{-}idem}(\mathcal{D}) = \emptyset$*, recovery preserves the retained logical cut subject to the version/epoch protocol. The stronger* $\text{Recover}(\Gamma_{crash}) \approx_{\text{obs}} \mathsf{Commit}(\Gamma)$ *remains a proof obligation under matched admissible responses, authorized endpoint reconciliation, interference-stable resource/closure relations and an explicit schedule interpretation. Idempotence or receipt availability alone does not establish it.*

3. ***Part III (Conservative Unknown Safety and Proposed Directed Simulation):*** *recovery of an authorized non-idempotent raw or mutating adapter attempt without a result retains its original uncertainty and spent reservation, fabricates no observation and does not automatically retransmit it. Full* $\text{Recover}(\Gamma_{crash}) \preccurlyeq_{\text{ep}} \Gamma$ *additionally requires the peer/reconciliation obligations above. No outer exact-debit bisimulation after retry follows from hiding; expenditure, effects and authority/generation observations must be accounted for.*

Replay argument and recovery limits. Part I's historical induction uses determinism of the reducer, immutable programs/observations and the read-only replay guard. For Part II, retaining projections/handles/charges and changing activation authority yields the same local successor only for the same selected observation and valid authority. Volatile ready-run position and in-flight workers are not retained semantic state. Phase 2A found that re-exposure could bypass a fresh source or scope decision. Phase 2B gates every physical opportunity, including recovery of an earlier raw idempotent reservation. Selected pure process cuts do not prove the general simulation.

For Part III, unknown-attempt eligibility and settlement rules support no-repeat/no-fabrication safety. Lemma 4.4 permits enabled peers to progress despite an unmatched receive, not identical peer histories under shared competition. Surviving live late transport

and fresh-process unknown recovery are distinct environments. Neither late evidence nor abandonment/retry restores exact-debit equivalence merely by hiding. The general peer/reconciliation/retry simulation argument remains open.

Phase 2A process qualification. In W_3, U0 holds the worker before launch and U1 holds its genuine response after endpoint commit, before local recording. These independent counter histories disagree on application. Neither selected cut kills an in-flight packet. Recovery sees the same original request, key, permission, reservation and local records. With a responsive exact-request idempotent endpoint the raw route applies for the first time at U0 or obtains the existing receipt at U1, then the live consumer selects it. A non-idempotent raw attempt and a previously reserved adapter attempt remain unresolved without another launch. In a separate case the endpoint commits and closes the connection before replying. The adapter retains an unsuccessful observation, the child selects typed defaults, and its guarded failure can reach the parent. The missing-result report clears while the transport error leaves remote application unresolved. A deliberately marked unsuccessful candidate can instead preserve an authentic receipt in its raw response. The ordinary parent sees only its selected invocation envelope. Its issued request supports conservative awareness of a possible effect. The failure envelope alone cannot decide application or authorize a retry. General future simulation and effectful compositional recovery remain open.

Phase 2B recovery and delayed application. Opening a supported historical Store preserves its replay interpretation and exact original request bytes. Every future physical exposure uses the current gate. Admission tokens disappear at process death. At the paired open cuts a responsive keyed raw endpoint permits useful same-operation recovery. The non-idempotent raw and already reserved adapter routes withhold another call. When the original source is revoked, the two keyed branches can yield the same successful local quiescence acknowledgment with equal retained evidence and different application histories.

In the delayed-packet experiment an independent endpoint acknowledged the original request before application. An authorized operation revoked its source, the executor was killed, and a fresh owner obtained Q. The endpoint then applied the held request under its declared client-independent service contract. The host can therefore observe local drain while external application is still possible. An ordinary parent selects its original invocation envelope. A failed child can publish the same failure whether a fresh admission was refused or its previously admitted effect subsequently applied. The host acknowledgment itself never enters authored state. Q can settle genuine surviving evidence for its original source. An open member of a revoked scope can still select that observation through its ordinary receive. Justifying a resolving action for the original uncertain operation remains a separate authority and endpoint question.

Phase 2C conditional original-operation recovery. Fix the single-query or absence-then-found schedule in which the final reported lookup first adopts a missing success-policy counter result. It requires an exact unique application visible on an unchanged trusted ledger, a separately entitled live resolver, successful original R/Q, an available authorized read or retained evidence, and actual accounting and message service. Atomic retention links the terminal lookup, its import decision and the missing original's tagged reconstruction. The separate original settlement, aggregate publication, resolver observation and caller selections can then proceed. The caller checks the original issuance and correlated found receipt, including its mutation and total. If the first of two queries already adopts, the last reports a duplicate and this particular caller rejects it despite the earlier adoption. A retained unsuccessful interaction instead permits a later report of application alongside the original failure. Raw originals and unsupported adapter policies have no success reconstruction under this rule.

With an accepted original packet still in flight, a fresh owner can obtain original Q and query an empty snapshot before that same packet applies. Losing the query response before retention permits a newly admitted read of a later snapshot on the remaining ordinal. Retaining the absence requires a distinct logical query to refresh. Two owner losses before transmission can consume both opportunities without one endpoint read. Replay consumes neither. These histories use a new evidence channel unavailable in the earlier equal-local-evidence counterexample. They establish conditional progress in this environment. General effectful composition, simulation under shared costs and authority, and evidence of definitive nonexecution remain open. The model and process histories are archived in `docs/foundational/phase2c`.

Phase 2D recovery costs and completion. Conditional progress requires both originals to reach physical admission before issuer closure, unique applications visible at the relied-on snapshots, original revocation and local drain, intact read and import authority, remaining opportunities, working storage and actual service. With capacity one, a held left worker can outlive the right issuer's handshake and End. The right request then lacks admission despite sufficient semantic allowance. Even a later application cannot revise an earlier retained absence. One sequential resolver per original ensures that selecting its importing query follows original and query settlement. A different, already admitted reader can select a duplicate while the importer still owes original settlement and the original invocation has published.

In the joined, successfully accounted automatic suffix, the prior invocation deliveries and message acknowledgments have occurred. The closed tree and designated programs provide no lifecycle work or terminal subscribers. Each new final acknowledgment is thus the sole internal delivery. The unchanged runner services and acknowledges it before committing another ready instruction, while its sender and recipient are open. The same reasoning applies on reopening exactly after that Send and before its first service. The resolvers still have to select their acknowledgments and terminate. A manual driver may instead commit the caller's End first, leaving an open resolver after a correct decision.

The comparison records invocation and member charges, mutation and logical-read debits, physical opportunities, mutation and lookup contacts, and business applications. A matched durable prefix with the same ordered applications can reach the same projected role outcomes after a lost lookup response, spending an extra ordinal and lookup contact. An opportunity spent before transmission can instead cost an ordinal with zero contact. These conditional claims concern the explicit single-host frame. The general symmetric simulation and the earlier impossibility without a new evidence channel retain their original scope. See `docs/foundational/phase2d/02-frame-and-completion.md`.

Under an endpoint contract that deduplicates identical immutable keys and parameters, execution-scoped keys support at-most-once logical mutation within a generation across permitted physical retries. This endpoint-dependent safety fact is separate from the open future-simulation claim and does not authorize retransmission after closure.

Theorem 6.3 (Impossibility of Blind Cross-Generation Business Deduplication)**.** *Across a supervisor-initiated* ***`Retry`*** *from $x_1 = (a, g)$ to $x_2 = (a, g+1)$, the execution-scoped external keys* $\mathsf{ExtKey}(\mathcal{N}_{256}, (a, g), n, 0) \neq \mathsf{ExtKey}(\mathcal{N}_{256}, (a, g+1), n, 0)$ *are distinct. Consequently, computation recovery and outbox idempotence cannot prevent duplicate external mutation across generations unless the application threads a generation-invariant business key or reconciles* Unknown *prior to retry (witnessed by* ***`business:order-7`*** *in Phase 1C).*

6.3 Two-Agent Interrupted Delegation: the Phase 3A Custody Boundary

Phase 3 asks whether local guarantees plus an interaction contract establish a system guarantee for two independently progressing durable agents. Phase 3A starts with one fixed delegation d from A to B, an immutable request and grant, two possible opaque authentic results,

asynchronous one-slot request/result/ACK channels, and at most one interruption of A. B's application and result retention are atomic by assumption. This independent finite model does not qualify a distributed implementation or solve the earlier external effect/receipt gap.

Weak local contracts and counterexample. A preserves its committed request, any durably recorded result and once-only consumption marker; volatile input may be lost. B applies d at most once, retains the result until delivery ACK, and keeps a deduplication tombstone after release. Consider: B applies and retains r; A receives r only in volatile memory and sends ACK; B receives ACK and releases r; with no result message in flight, A crashes before recording r and recovers. Both local contracts hold, but useful recovery of the actual result fails. The contracts do not imply continuous durable responsibility for the result.

Indistinguishability boundary. Choose $r_1 \neq r_2$ and the same loss schedule. Assume the tombstone retains neither result nor distinguishing digest, and no archive, lookup or regeneration channel is available. The two recovered agent/channel states coincide while the inaccessible actual results differ. A recovery procedure restricted to that state cannot guarantee the authentic actual result in both worlds. Withholding is safe but does not meet useful recovery. This is a scoped information-loss argument, not an unrestricted distributed impossibility result.

Conditional custody lemma. Let D_A, D_B denote retained copies of the original result, with A retaining D_A after consumption. Suppose application establishes $D_B = r$; only authentic d-bound copies populate D_A; committed D_A survives interruption and is never deleted; and B releases only on an authentic ACK causally after A's durable record. Then every reachable prefix after application satisfies

$$D_B = r \ \vee \ D_A = r.$$

Application establishes the left disjunct. Transmission, delivery and interruption do not remove a durable copy. Recording establishes the right disjunct; consumption preserves it. Before release, the ACK precondition supplies the right disjunct. This local case argument composes preservation guarantees through one edge condition. It proves custody safety under the stated assumptions, not general compositional recovery. Completion reachability in a finite graph is a separate existential check; inevitable progress needs recovery and service of request, application, retransmission, delivery, record and consumption work.

The distinction between delivery acknowledgment and application responsibility follows the established end-to-end argument of Saltzer, Reed and Clark [17]. The experiment makes that distinction explicit in ALAN's interaction obligations; it does not claim the principle is new. The current Store already records accepted observations in the admission transaction and consumption in its commit transaction, so the weak delivery ACK is not a demonstrated Store defect. This one-edge slice leaves independent-owner process correspondence, B crashes, grant attenuation, cancellation, mutual waits and arbitrary-size closure open; the next subsection addresses a restricted reciprocal case. See `docs/foundational/phase3a/03-results.md`.

6.4 Reciprocal Obligations: the Phase 3B.1 Service Boundary

The next slice gives each of two actual agents one outgoing original obligation and the other agent's incoming obligation. Each edge uses durable ACK. A crash disables both roles of that agent, clears its volatile received result and preserves both durable roles and channels; recovery changes activation, not original identity. The finite model allows one crash per agent, two opaque results per edge, retries and duplicate delivery. Fixed disjoint grants and resources, authentic channels, intact storage, and atomic application with receipt retention remain assumptions.

Write c_e, a_e, u_e for durable commitment, application and consumption of original edge e. Agent i is waiting when its outgoing request is committed but not consumed. A serial agent

permits incoming application only when it is not waiting; a servicing agent permits it while waiting. The guard restricts application, not admission, receipt transmission, recording, consumption or ACK handling by an up agent. Define

$$\mathsf{Complete} = u_{AB} \wedge u_{BA}, \qquad \mathsf{Trap} = c_{AB} \wedge c_{BA} \wedge \neg a_{AB} \wedge \neg a_{BA}.$$

Here $\mathsf{CanComplete}(s)$ means that some finite legal suffix reaches Complete, not that all executions terminate.

Conditional two-edge proposition. In this declared transition system, custody holds on both edges in all four serial/servicing profiles. For two serial agents, $\neg\mathsf{CanComplete}(s)$ is equivalent to Trap at every reachable state. If either agent is servicing, every reachable state has a completion suffix.

Proof. Project either edge onto the Phase 3A durable-ACK model, consistently renaming its fixed identities. Its caller's crash/recovery projects to the corresponding one-edge step; callee crash/recovery and work on the opposite edge project to stutters. Shared availability and the service guard remove possible steps without adding invalid projected steps. The one-edge custody induction therefore applies twice. This safety argument does not lift completion through the removal of enabled work.

In serial Trap, a first application would require its callee to have consumed the opposite original's result, which requires an earlier opposite application: a contradiction. Recovery preserves those guard facts. Outside Trap, if either original has applied, custody supplies a path to consuming it; its caller's serial incoming-service gate then opens for the other original. If neither has applied, some original is uncommitted: keep it uncommitted while that agent serves the other request, then use the preceding case. Recover down agents first and choose a suffix without further crashes. If A is servicing, it can serve B's original irrespective of its own outstanding request; B consumes that result, opening B's serial gate for A's original. Omit already completed steps. The symmetric case and two servicing agents follow. These are conditional source-level arguments, not mechanized proofs or arbitrary-program theorems.

The frozen graphs contain 68,497 states for serial/serial and 96,721 for each other profile, with zero local-safety or custody violations. Exactly 144 serial/serial states lack a completion suffix; every state in each other profile has one. The shortest serial Trap prefix has two commitment events. A distinct six-event witness admits both requests with both agents up and all channels empty; ten events reach that admitted wait after both agents crash and recover. Retry cycles remain enabled, so this is a completion obstruction rather than a zero-outdegree graph state. Inevitable progress outside the obstruction requires eventual recovery and actual service of the necessary actions; an unfair retry cycle can postpone useful work forever. The finite graph checks and retained traces are in `docs/foundational/phase3b/03-results.md`.

A separate public-runtime fixture uses ordinary authored roots in one Store, with final unsupplied hold inputs that keep both sources open. Both original Sends, request admissions and delivery acknowledgments precede ordinary Run. Across both Send orders and both delivery orders, serial/serial leaves two admitted requests unconsumed; either servicing root enables consumption of both validated replies. Repeated Run and whole-Store reopen preserve the serial wait while reopening renews both activation epochs. The serial Store correctly reports quiescence with zero pending deliveries and zero UnresolvedIntents, yet owes two application replies. Those bookkeeping fields do not certify application completion. Runtime delivery ACKs also differ from the model's receipt-release ACK protocol. This is a bounded control-order witness, not independent-owner recovery, external-effect qualification, full model/runtime refinement or root termination.

Phase 3B.1 covers this reciprocal service dependency and shared crash availability in the model. Dynamic capability attenuation, cancellation, non-atomic world effects, distributed

ownership, resource exhaustion and arbitrary-size closure remain open. The custody projection provides a local safety proof pattern; it does not establish associativity or discharge new service cycles introduced by further agents.

7 Conclusion

If a parent was waiting for a child invocation when its process died, a fresh process can recover the identity of the call and the instruction that awaits it. Reopening the store preserves the logical run and the resources it has already spent. An ordinary supervisor can receive a child's authenticated terminal outcome, consult its own retry budget and request another attempt under a lifecycle grant. An accepted retry starts a new generation.

Consider a parent that invokes two children with the same input. The right child's result can wait in the parent's inbox while the parent waits for the outcome of its left call. The proof follows the identities of these calls through finite trees of joins, relays and forks. At each leaf it allows either the direct program or its helper-based replacement. Given the specified inputs, an open scope, sufficient resources and service of eligible work, every completed tree returns each answer or diagnosis in its proper position. A leaf that stays pending keeps its ancestors from publishing a result. Calling the helper costs an extra invocation even when the two programs give the same answer.

Now suppose a counter increments just before the issuing process dies. If the receipt was never recorded, the surviving local evidence is also consistent with a request lost in transit. Repeating the request can increment the counter twice. The runtime keeps the non-idempotent attempt unresolved and withholds retransmission. In the controlled construction, separately authorized readers can obtain unique matching receipts from the counter's own ledger. The parent can then combine evidence for two original mutations with the result of a pure tree, preserving an earlier interaction failure alongside later evidence of application. A lookup that finds no entry cannot rule out an increment from a packet still in flight.

Preserving committed state and result custody does not ensure that two agents can produce the results they owe one another. In the reciprocal model, each serial agent waits for its own result before applying its peer's request. Durable ACK preserves custody but cannot break that dependency. Allowing either agent to serve while waiting supplies a completion suffix under the declared assumptions. The same-Store fixture retains the corresponding wait across reopening; quiescence alone does not pay the application obligations.

To extend substitution and recovery to arbitrary contexts, a proof must account for peers that can spend a shared allowance, close a scope or reach a shared endpoint between two matched steps. Even when recovered and uninterrupted runs reach the same public outcome, recovery can cost another lookup. On a single host the store orders revocation against admission to dispatch. How to preserve that ordering when authority passes between hosts and either host can crash remains open.

References

[1] Gul Agha, Ian A. Mason, Scott Smith, and Carolyn Talcott. A foundation for actor computation. *Journal of Functional Programming*, 7(1):1–72, 1997.

[2] Nuel D. Belnap, Jr. A useful four-valued logic. In J. Michael Dunn and George Epstein, editors, *Modern Uses of Multiple-Valued Logic*, pages 5–37. D. Reidel, 1977.

[3] Mike Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 335–350, 2006.

[4] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems (TOCS)*, 3(1):63–75, 1985.

[5] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936.

[6] Matthias Felleisen. On the expressive power of programming languages. *Science of Computer Programming*, 17(1–3):35–75, 1991.

[7] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI '73)*, pages 235–245, 1973.

[8] C. B. Jones. Tentative steps toward a development method for interfering programs. *ACM Transactions on Programming Languages and Systems*, 5(4):596–619, 1983.

[9] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[10] Robin Milner, Joachim Parrow, and David Walker. A calculus of mobile processes, I and II. *Information and Computation*, 100(1):1–77, 1992.

[11] Marvin L. Minsky. Recursive unsolvability of Post's problem of "Tag" and other topics in theory of Turing machines. *Annals of Mathematics*, 74(3):437–455, 1961.

[12] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[13] Peter W. O'Hearn. Resources, concurrency, and local reasoning. *Theoretical Computer Science*, 375(1–3):271–307, 2007.

[14] Andrew M. Pitts. Nominal logic, a first order theory of names and binding. *Information and Computation*, 186(2):165–193, 2003.

[15] Gordon D. Plotkin. A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60–61:17–139, 2004. Originally published as Technical Report DAIMI FN-19, Aarhus University, 1981.

[16] Gordon D. Plotkin and Matija Pretnar. Handling algebraic effects. *Logical Methods in Computer Science*, 9(4:23):1–36, 2013. Conference version in ESOP 2009, LNCS 5502, pp. 80–94.

[17] Jerome H. Saltzer, David P. Reed, and David D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.

[18] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, 1990.

[19] Daniele Turi and Gordon D. Plotkin. Towards a mathematical operational semantics. In *Proceedings of the 12th Annual IEEE Symposium on Logic in Computer Science (LICS '97)*, pages 280–291, 1997.

[20] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1936.

[21] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. Interaction trees: Representing recursive and impure programs in Coq. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–32, 2020.